

생성형 AI 이해와 ChatGPT API 활용 App 개발

실습 환경 구성

Anaconda 설치

Download from <https://www.anaconda.com/download/success>

Download Now

Download Anaconda Distribution or Miniconda by choosing the proper installer for your machine. Learn the difference from our [Documentation](#).

Distribution Installers

 Download

For installation assistance, refer to [troubleshooting](#).

Miniconda Installers

 Download

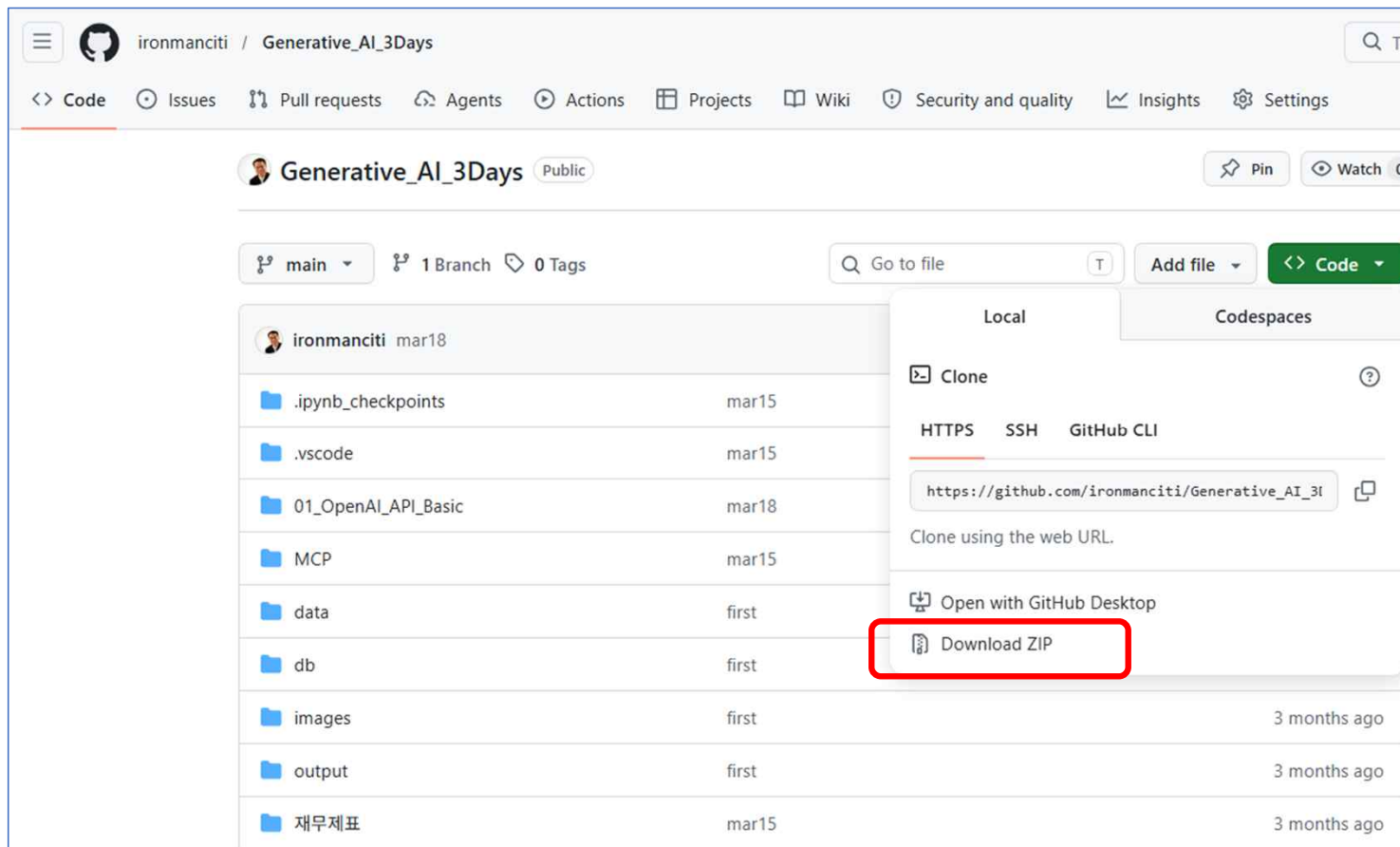
For installation assistance, refer to [troubleshooting](#).

Google Colaboratory 소개

- Free GPU 제공
 - Google Drive 와 연동
 - Jupyter Notebook 환경
 - Deep Learning beginner 를 위한 최적의 환경
 - 각종 snippet 제공
-

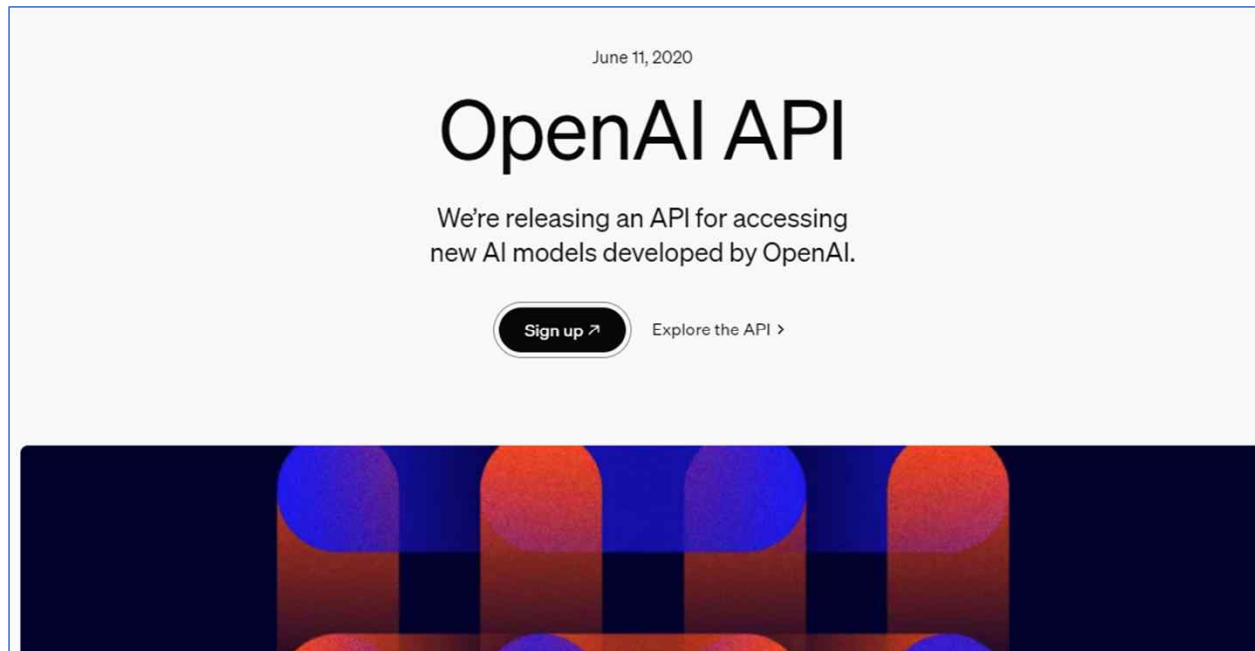
Github Repository 에서 실습 code download

https://github.com/ironmanciti/Generative_AI_3Days



Open-AI 계정 등록

<https://openai.com/index/openai-api/>



계정 만들기

이메일 주소*

[계속](#)

이미 계정이 있으신가요? [로그인](#)

_____ 또는 _____

 Google로 계속하기

 Microsoft 계정으로 계속하기

 Apple로 계속하기



Search CTRL K

GET STARTED

Overview

Quickstart

Models

Pricing

Changelog

Terms and policies

CAPABILITIES

Text generation

Vision

Image generation

Audio generation

Text to speech

OpenAI developer platform

Developer quickstart

Set up your environment and make your first API request in minutes

5 min

python

```
1 from openai import OpenAI
2 client = OpenAI()
3 completion = client.chat.completions.create(
4     model="gpt-4o",
5     store=True,
6     messages=[
7         {"role": "user", "content": "write a haiku about ai"}
8     ]
9 )
```



Meet the models

Pricing

Billing 정보 등록 (Settings → Billing)

P

Personal

/

Default project

Playground

Dashboard

Docs

API reference





SETTINGS

Your profile

ORGANIZATION

General

API keys

Admin keys

Members

Projects

Billing

Limits

Usage

Data controls

Verifications

Billing

Overview

Payment methods

Billing history

Preferences

Pay as you go

Credit balance ⓘ

\$10.50

Add to credit balance

Cancel plan

✓ Auto recharge is on

When your credit balance reaches \$5.00, your payment method will be charged to bring the balance up to \$15.00.

Modify

 Payment methods

Add or change payment method

 Billing history

View past and current invoices

사용한 만큼 지불하는 방식 (Pay as you go)

• 자동 충전 설정: 켜져 있으며, 크레딧 잔액이 \$5.00에 도달하면 결제 방식이 사용되어 잔액이 \$15.00로 충전됩니다.

API key 생성 (Dashboard → API keys) <https://platform.openai.com/api-keys>

P Personal / Default project

Playground Dashboard Docs API reference

DASHBOARD

Chat Completions

Assistants

Batches

Evaluations

Fine-tuning

Storage

Usage

API keys

API keys

+ Create new secret key

As an owner of this project, you can view and manage all API keys in this project.

Do not share your API key with others or expose it in the browser or other client-side code. To protect your account's security, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

NAME	SECRET KEY	LAST USED	CREATED BY	PERMISSIONS
My Test Key	sk-...7XGD	Never	YoungJea Oh	All

이 프로젝트의 소유자로서 귀하는 이 프로젝트의 모든 API 키를 보고 관리할 수 있습니다.

API 키를 다른 사람과 공유하거나 브라우저 또는 기타 클라이언트 측 코드에 노출하지 마십시오. 귀하의 계정 보안을 보호하기 위해 OpenAI는 공개적으로 유출된 모든 API 키를 자동으로 비활성화할 수도 있습니다.

[사용량 페이지](#)에서 API 키별 사용량을 확인하세요.

Key는 한번만 보여주므로 메모장 등에 복사한 후 .env 파일로 저장

Create new secret key

Owned by

☒ You ☐ Service account

이 API 키는 사용자와 연결되어 있으며 선택한 프로젝트에 대해 요청할 수 있습니다. 조직이 나 프로젝트에서 제거되면 이 키가 비활성화됩니다.

Name Optional

My Test Key

Project

Default project

Permissions

All Restricted Read Only

Cancel Create secret key

Save your key

Please save this secret key somewhere safe and accessible. For security reasons, **you won't be able to view it again** through your OpenAI account. If you lose this secret key, you'll need to generate a new one.

3RuDSd7TQbDNViu39T3qT3B1bkFJEoTJIwcT8PhptPA

Copy

Permissions

Read and write API resources

Done

이 프로젝트의 소유자로서 귀하는 이 프로젝트의 모든 API 키를 보고 관리할 수 있습니다.

API 키를 다른 사람과 공유하거나 브라우저 또는 기타 클라이언트 측 코드에 노출하지 마십시오. 귀하의 계정 보안을 보호하기 위해 OpenAI는 공개적으로 유출된 모든 API 키를 자동으로 비활성화할 수도 있습니다.

사용량 페이지에서 API 키별 사용량을 확인하세요.

이름	비밀키	작성자	권한
내 테스트 키	sk-...7XGD	오영재	모두

생성된 API Key 관리

- 타인 노출 안되도록 조심
- 노출되면 반드시 삭제 후 재 생성

API keys

+ Create new secret key

As an owner of this project, you can view and manage all API keys in this project.

Do not share your API key with others or expose it in the browser or other client-side code. To protect your account's security, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

NAME	SECRET KEY	LAST USED ⓘ	CREATED BY	PERMISSIONS
My Test Key	sk- . . . 7XGD	Never	YoungJea Oh	All

단일 프로젝트에 대해 API key 설정 방법

- .env – API 키가 포함된 로컬 파일 생성

```
# .env 파일  
OPENAI_API_KEY=sk-vvDtI*****XiilEpdjLhBJaH0f
```

- .gitignore 에 .env 파일 포함

- Python code

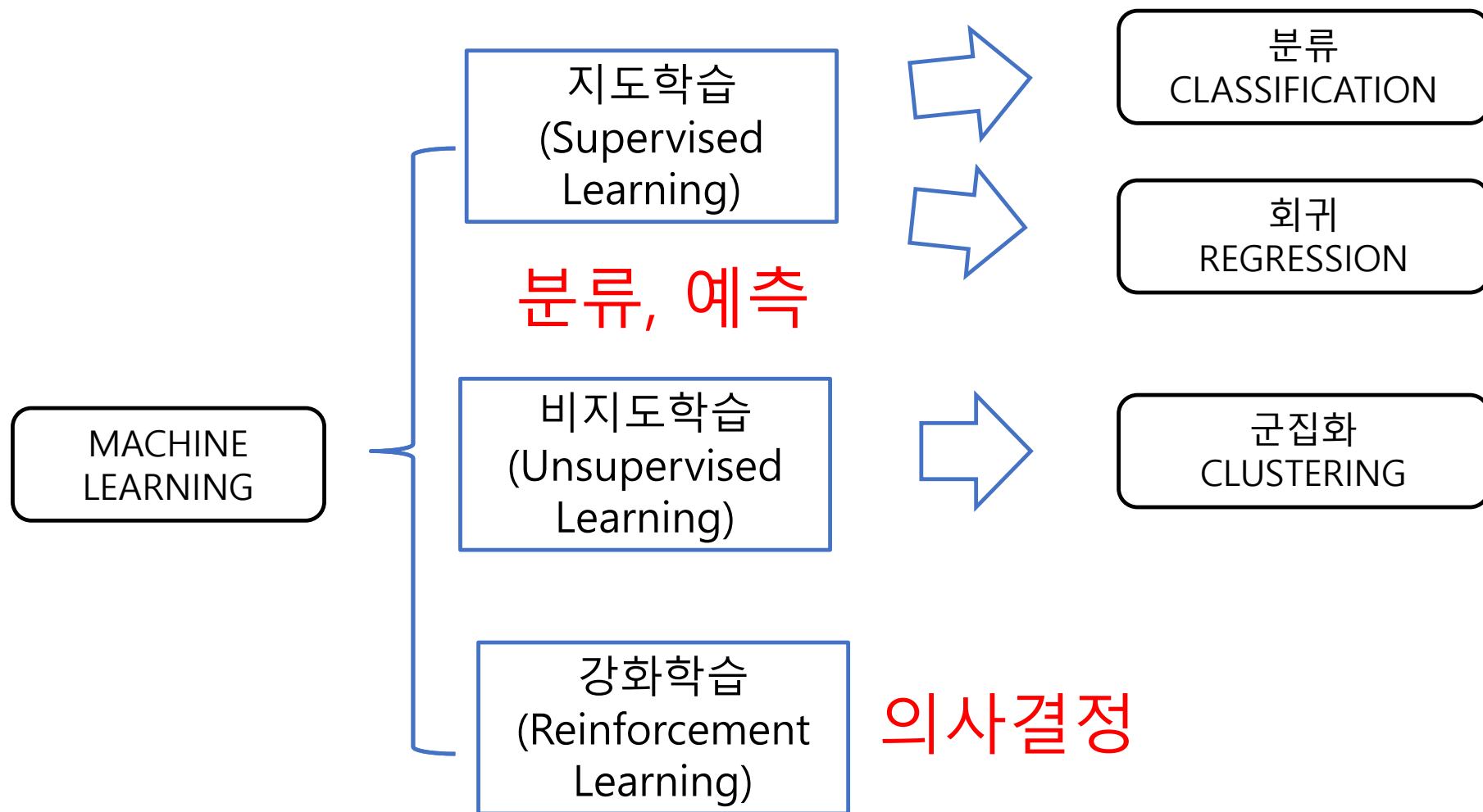
```
pip install python-dotenv
```

```
from dotenv import load_dotenv,  
_ = load_dotenv() # local .env file을 읽어서 os.environ 에 OPENAI_API_KEY 추가
```

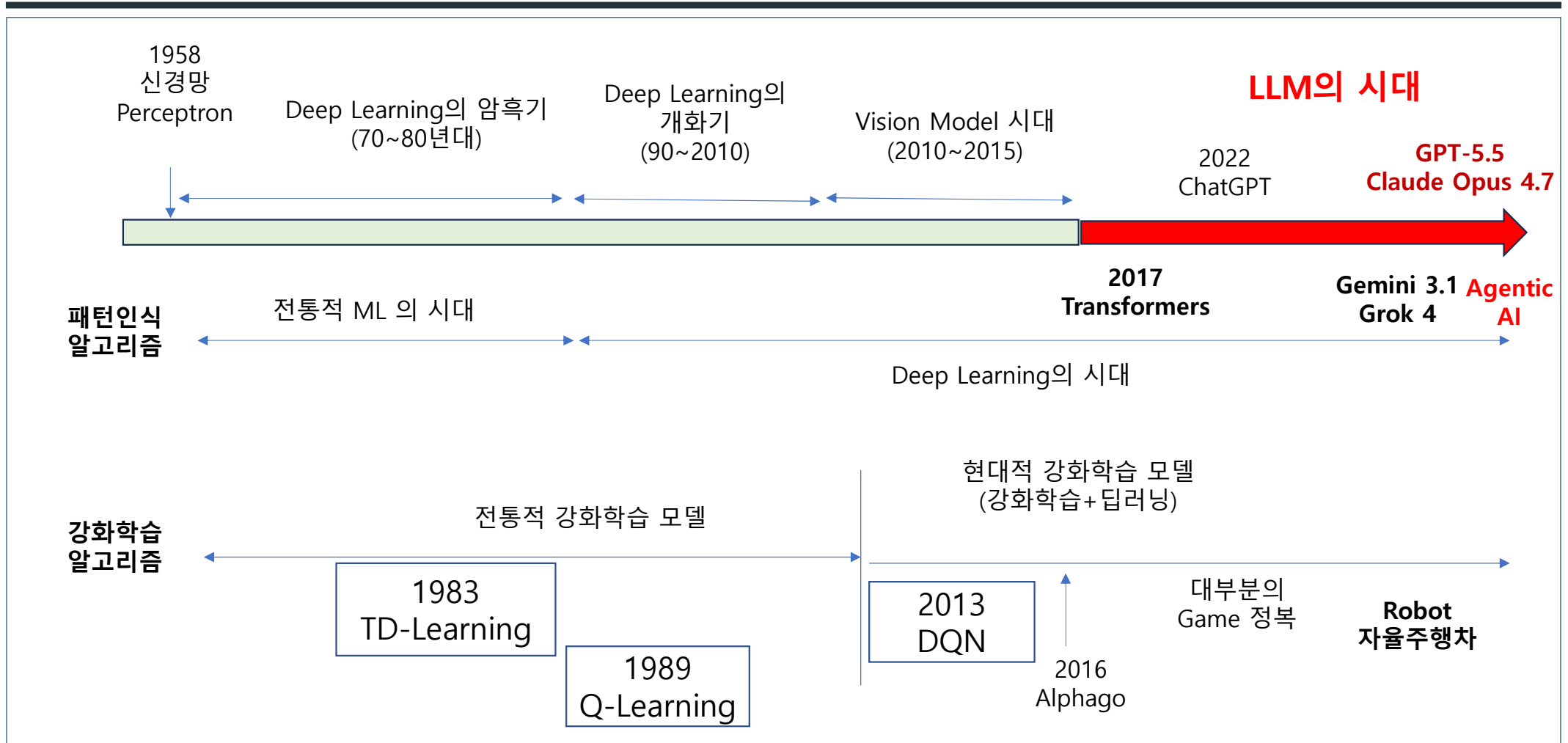
```
from openai import OpenAI  
client = OpenAI()
```

생성형 AI 개요

Machine Learning (AI)의 종류



AI 발전 History

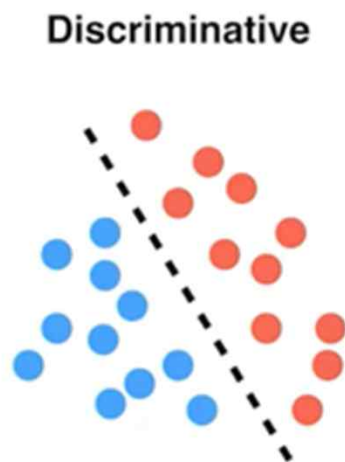


생성 모델이란 ?

- 실제 존재하지는 않지만 있을 법한 이미지를 생성할 수 있는 모델

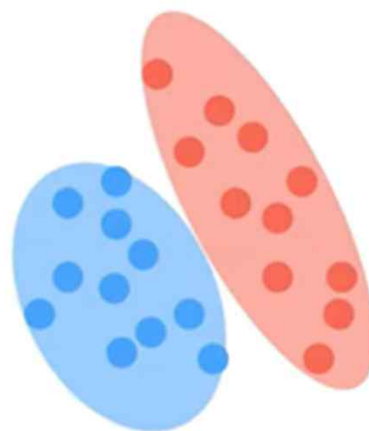
- 분류/예측/군집화
- Feature와 label 간의 관계 학습

분류 모델



결정 경계(Decision Boundary)를 학습

Generative



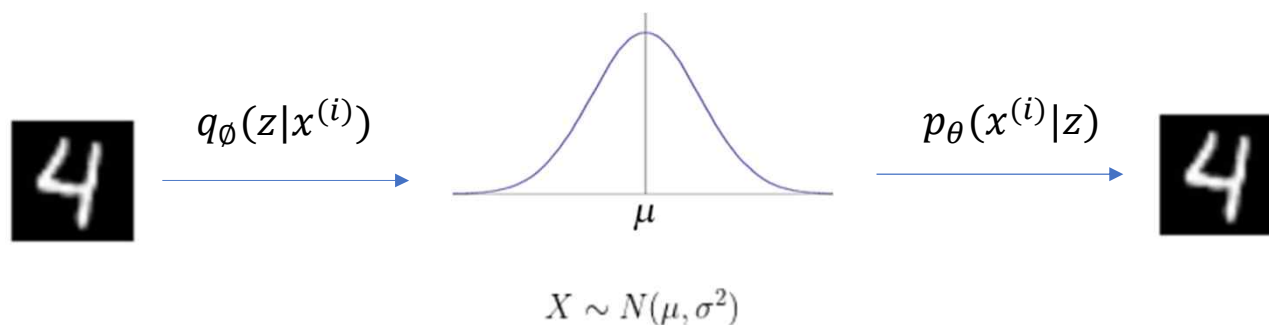
- 학습했던 데이터와 유사한 데이터 생성
- 데이터의 분포 학습
- 다음 단어 예측

생성 모델

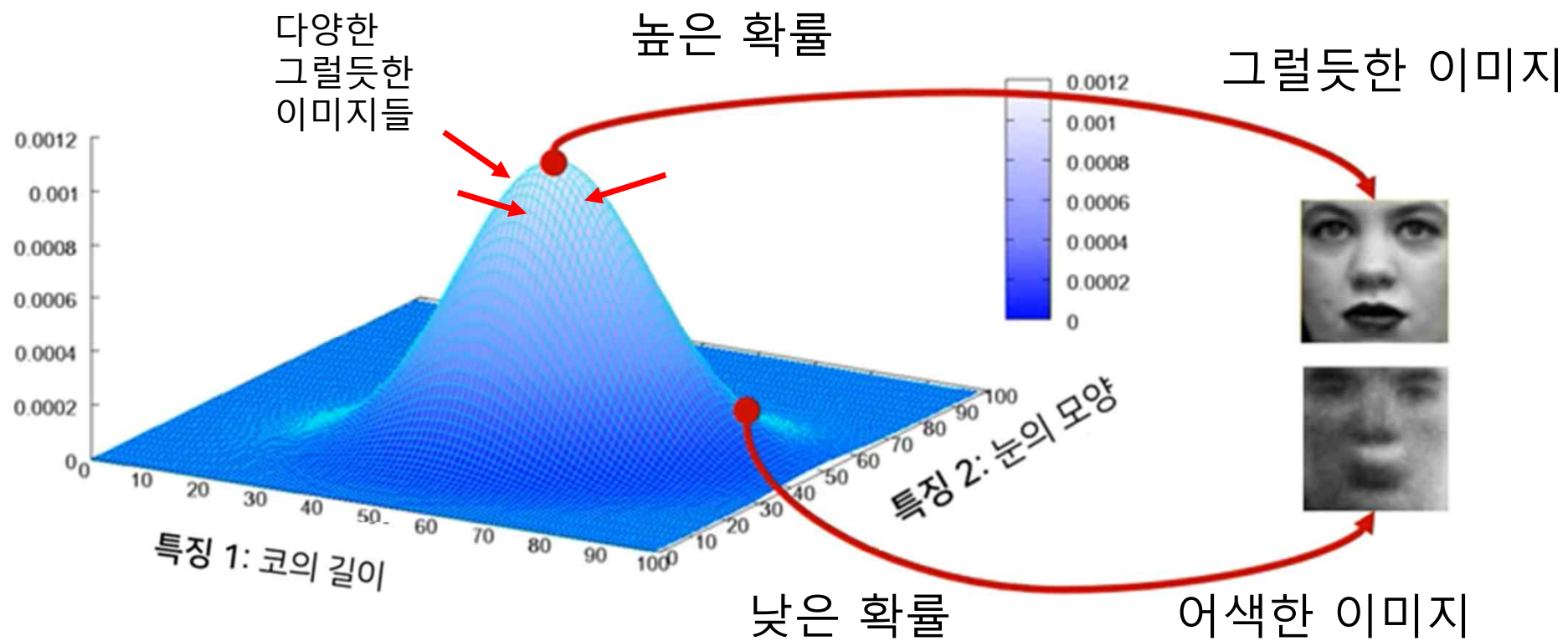
각 클래스의 분포(Distribution)를 학습

생성 모델(Generative Model)이란 ?

- Training data가 주어졌을 때 이 training data가 가지는 real 분포와 같은 분포에서 sampling된 값으로 new data를 생성하는 model
- 입력 $\rightarrow$ 잠재변수 분포를 생성 $q_{\phi}(z|x^{(i)})$
- 잠재변수로부터 sampling $\rightarrow$ 입력에 가까운 출력 생성 $p_{\theta}(x^{(i)}|z)$



결합확률(Joint Probability)을 학습



생성모델의 5대 계열

계열	핵심 원리	대표 모델	특징
VAE (2013)	입력→잠재변수 분포 학습 후 디코더로 복원	Stable Diffusion의 잠재공간	안정적 학습, 출력이 다소 흐릿
GAN (2014)	생성자 vs 판별자의 적대적 경쟁 학습	StyleGAN, 딥페이크	선명한 출력, 학습 불안정
Autoregressive (2017~)	다음 토큰의 확률을 순차적으로 예측	GPT 계열, GPT-Image	텍스트·코드 생성의 주류
Diffusion (2020~)	노이즈 추가(Forward)→제거(Reverse) 역과정 학습	DALL-E, Stable Diffusion, Sora, Veo	이미지·비디오 생성의 주류

현재의 주류: 텍스트 = Autoregressive(GPT), 이미지·비디오·오디오 = Diffusion

생성형 AI의 활용

- Chatbot
 - 가짜 Image 생성 (GAN)
 - Text 로 부터 이미지 생성
 - Code 생성
 - Voice 생성
 - Video 생성, etc
-

생성형 언어 모델

Generative Language Model

Large Language 모델의 발전 과정과 모델 생태계

- **2017** – **Transformer** 논문 "Attention is all you need" (Google)
- **2018** – OpenAI GPT · Google BERT
- **2019~2020** – GPT-2 → GPT-3 (1,750억 파라미터)
- **2022.11** – ChatGPT (GPT-3의 대화형 버전)
- **2023** – GPT-4 · BARD · Llama2
- **2024.9** – **추론 모델 (Reasoning Model)** 등장 – o1, CoT 기반 추론·코딩 강점
- **2025~26** – GPT-5.5 · Claude Opus 4.7 · Gemini 3.1 – **Agentic AI 시대 본격화**

폐쇄형 (Closed / API)

- GPT-5.5 · Claude Opus 4.7 · Gemini 3.1
- 최고 성능 · API 과금 · 데이터 외부 경유

경량 모델 (SLM)

- gpt-5-nano / mini — 비용·지연 절감, 온디바이스

오픈 웨이트 (Open-weight)

- Llama · Mistral · Qwen · DeepSeek
- 자체 설치·파인튜닝 — 데이터 주권·보안에 적합

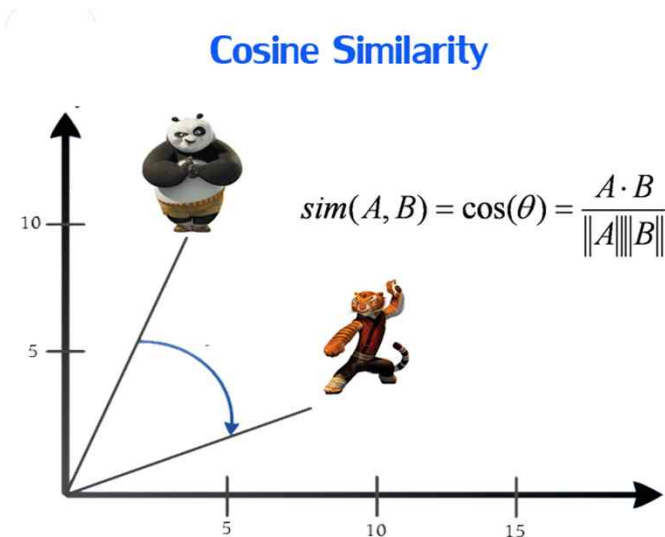
Embedding matrix (example)

words

	0	1	2	3	4	5	6	7	8	9	...	290	291	292	
fox	-0.348680	-0.077720	0.177750	-0.094953	-0.452890	0.237790	0.209440	0.037886	0.035064	0.899010	...	-0.283050	0.270240	-0.654800	0.105
ham	-0.773320	-0.282540	0.580760	0.841480	0.258540	0.585210	-0.021890	-0.463680	0.139070	0.658720	...	0.464470	0.481400	-0.829200	0.354
brown	-0.374120	-0.076264	0.109260	0.186620	0.029943	0.182700	-0.631980	0.133060	-0.128980	0.603430	...	-0.015404	0.392890	-0.034826	-0.720
beautiful	0.171200	0.534390	-0.348540	-0.097234	0.101800	-0.170860	0.295650	-0.041816	-0.516550	2.117200	...	-0.285540	0.104670	0.126310	0.120
jumps	-0.334840	0.215990	-0.350440	-0.260020	0.411070	0.154010	-0.386110	0.206380	0.386700	1.460500	...	-0.107030	-0.279480	-0.186200	-0.545
eggs	-0.417810	-0.035192	-0.126150	-0.215930	-0.669740	0.513250	-0.797090	-0.068611	0.634660	1.256300	...	-0.232860	-0.139740	-0.681080	-0.370
beans	-0.423290	-0.264500	0.200870	0.082187	0.066944	1.027600	-0.989140	-0.259950	0.145960	0.766450	...	0.048760	0.351680	-0.786260	-0.360
sky	0.312550	-0.303080	0.019587	-0.354940	0.100180	-0.141530	-0.514270	0.886110	-0.530540	1.556600	...	-0.667050	0.279110	0.500970	-0.270
bacon	-0.430730	-0.016025	0.484620	0.101390	-0.299200	0.761820	-0.353130	-0.325290	0.156730	0.873210	...	0.304240	0.413440	-0.540730	-0.030
breakfast	0.073378	0.227670	0.208420	-0.456790	-0.078219	0.601960	-0.024494	-0.467980	0.054627	2.283700	...	0.647710	0.373820	0.019931	-0.030
toast	0.130740	-0.193730	0.253270	0.090102	-0.272580	-0.030571	0.096945	-0.115060	0.484000	0.848380	...	0.142080	0.481910	0.045167	0.050
today	-0.156570	0.594890	-0.031445	-0.077586	0.278630	-0.509210	-0.066350	-0.081890	-0.047986	2.803600	...	-0.326580	-0.413380	0.367910	-0.260
blue	0.129450	0.036518	0.032298	-0.060034	0.399840	-0.103020	-0.507880	0.076630	-0.422920	0.815730	...	-0.501280	0.169010	0.548250	-0.310
green	-0.072368	0.233200	0.137260	-0.156630	0.248440	0.349870	-0.241700	-0.091426	-0.530150	1.341300	...	-0.405170	0.243570	0.437300	-0.460
kings	0.259230	-0.854690	0.360010	-0.642000	0.568530	-0.321420	0.173250	0.133030	-0.089720	1.528600	...	-0.470090	0.063743	-0.545210	-0.190
dog	-0.057120	0.052685	0.003026	-0.048517	0.007043	0.041856	-0.024704	-0.039783	0.009614	0.308416	...	0.003257	-0.036864	-0.043878	0.000
sausages	-0.174290	-0.064869	-0.046976	0.287420	-0.128150	0.647630	0.056315	-0.240440	-0.025094	0.502220	...	0.302240	0.195470	-0.653980	-0.290
lazy	-0.353320	-0.299710	-0.176230	-0.321940	-0.385640	0.586110	0.411160	-0.418680	0.073093	1.486500	...	0.402310	-0.038554	-0.288670	-0.240
love	0.139490	0.534530	-0.252470	-0.125650	0.048748	0.152440	0.199060	-0.065970	0.128830	2.055900	...	-0.124380	0.178440	-0.099469	0.000
quick	-0.445630	0.191510	-0.249210	0.465900	0.161950	0.212780	-0.046480	0.021170	0.417660	1.686900	...	-0.329460	0.421860	-0.039543	0.150
20 rows x 300 columns															

유사도 측정 (Cosine Similarity)

- 정의 - 고차원의 데이터를 저차원으로 매핑하면서, 그 의미를 보존하는 것
- 글, 이미지나 동영상같은 비정형 데이터를 숫자로 표현
- 큰 데이터를 저차원으로 압축하여 효율적으로 저장하며 의미 보존
- 인공지능 모델이 아주 잘 다룰 수 있는 표현 형식
- 데이터 간의 유사도 측정 방법
 - 코사인 유사도: -1~ 1 사이의 범위



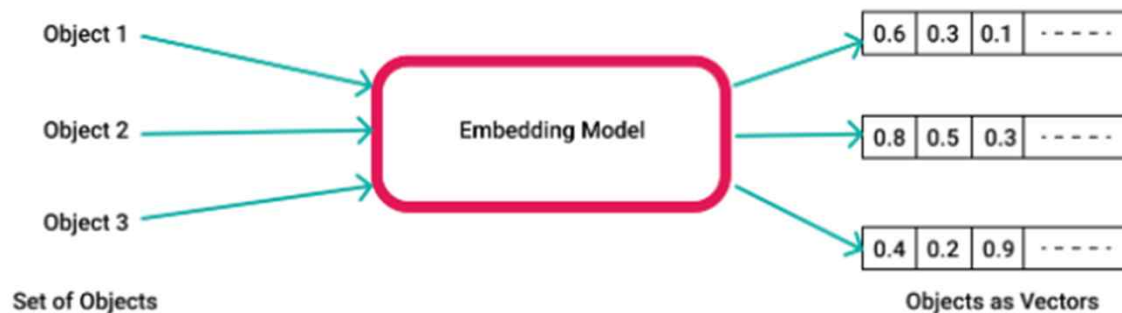
실습 : 010_embeddings

Embedding

OpenAI의 텍스트 임베딩은 텍스트 문자열의 관련성을 측정합니다. 임베딩은 일반적으로 다음 용도로 사용됩니다.

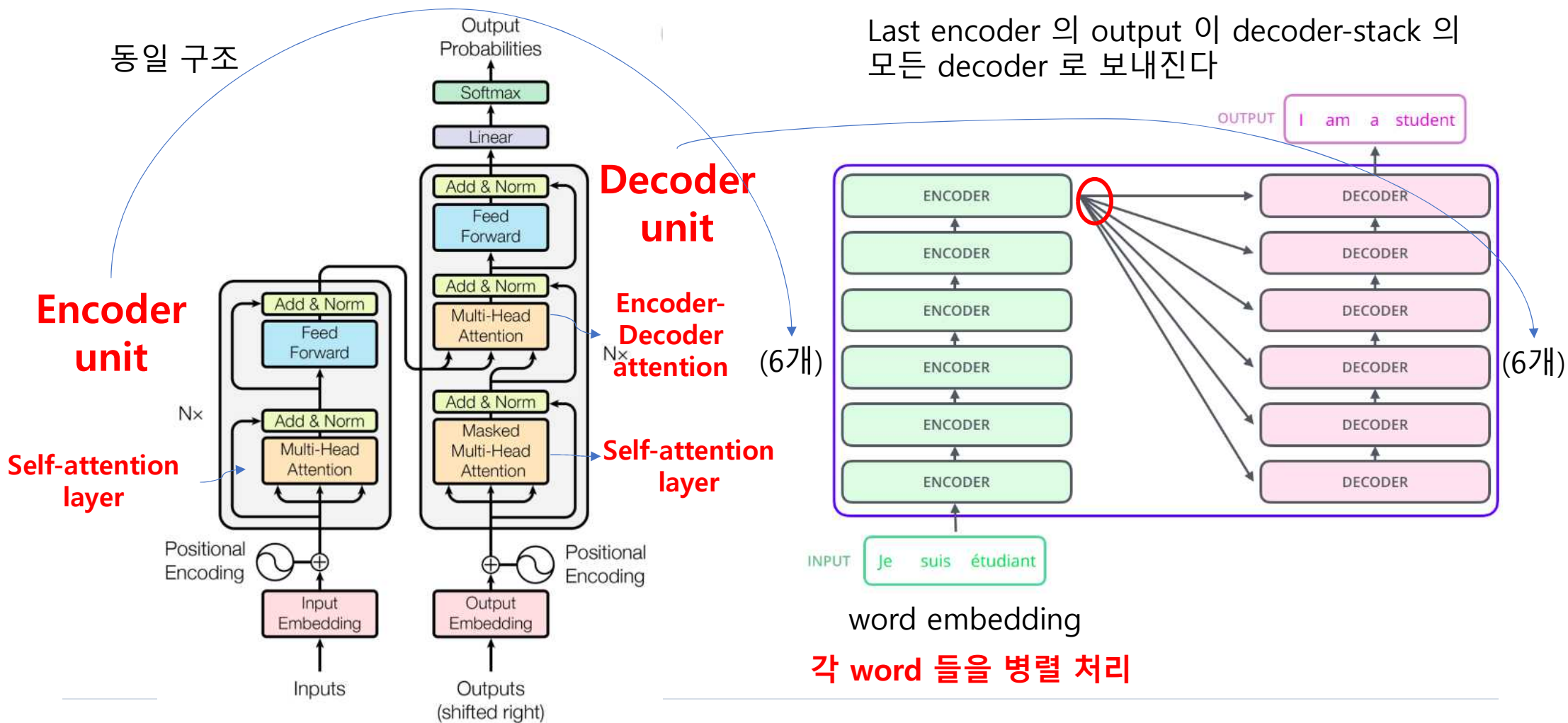
- 두 텍스트 사이의 관련성을 측정하는 데 사용할 수 있는 텍스트의 숫자 표현

임베딩은 부동 소수점 숫자로 표시되는 벡터입니다. 두 벡터 사이의 거리는 관련성을 측정합니다. 거리가 작을수록 관련성이 높음을 나타내고, 거리가 멀면 관련성이 낮다는 것을 나타냅니다.



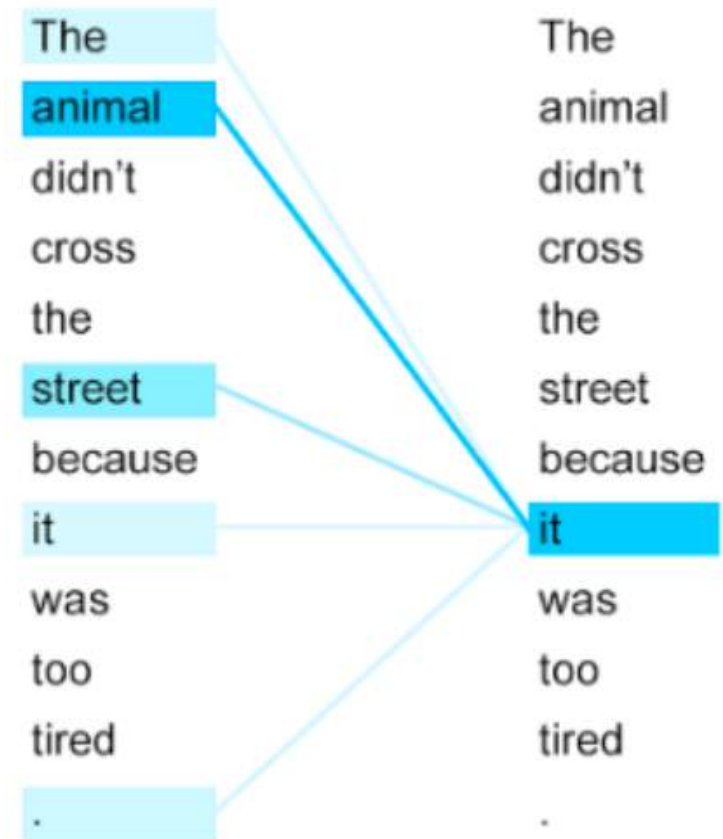
Transformers

Transformer Architecture

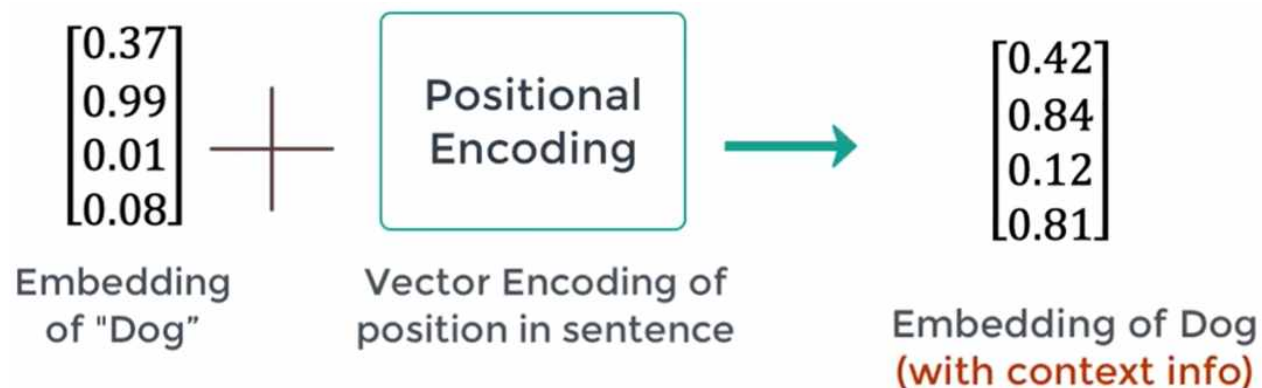
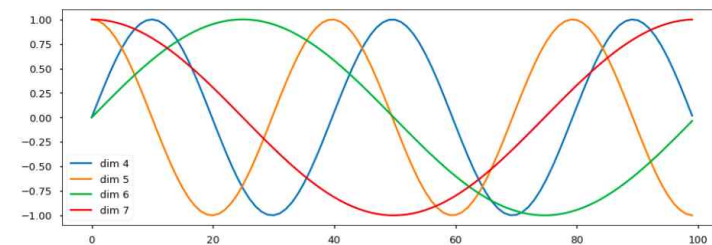
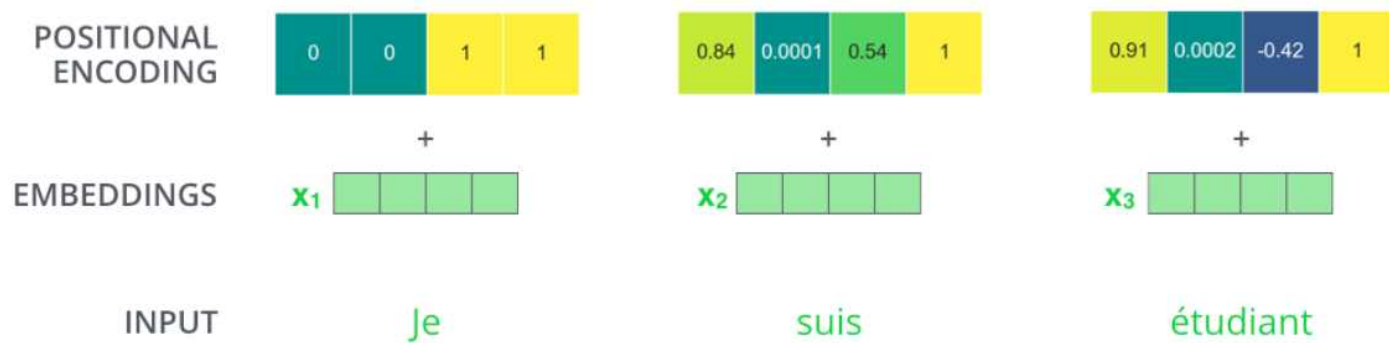


Self-Attention (Intra-Attention)

- Attention 을 자기 자신에 대해 수행
→ 문장 내 단어들 간의 유사도를 구함
- Self-attention 계산
3 개의 vector 필요 (훈련 과정에서 스스로 학습)
 - Query Vector
 - Key Vector
 - Value Vector

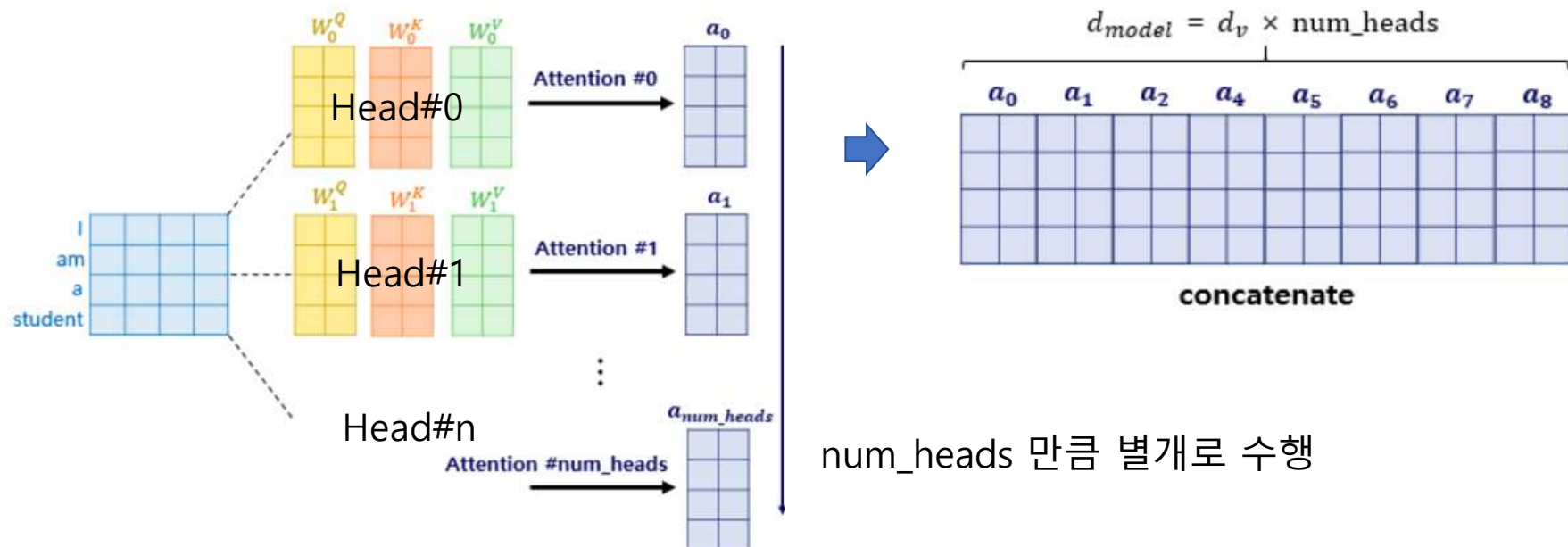


Positional Encoding example

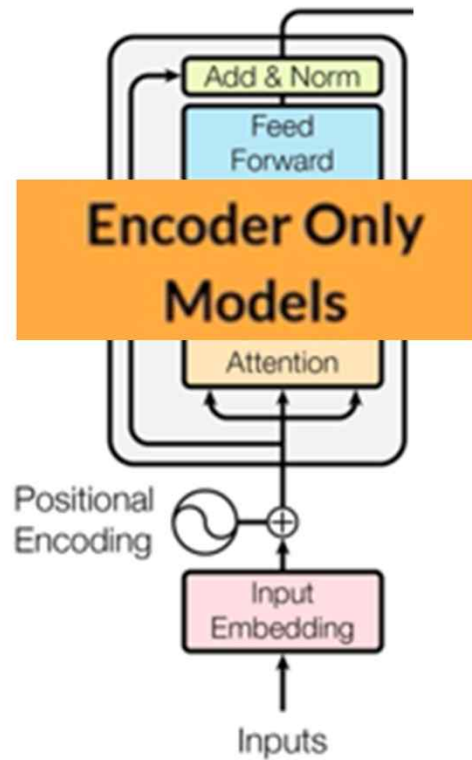


Multi-Head Attention

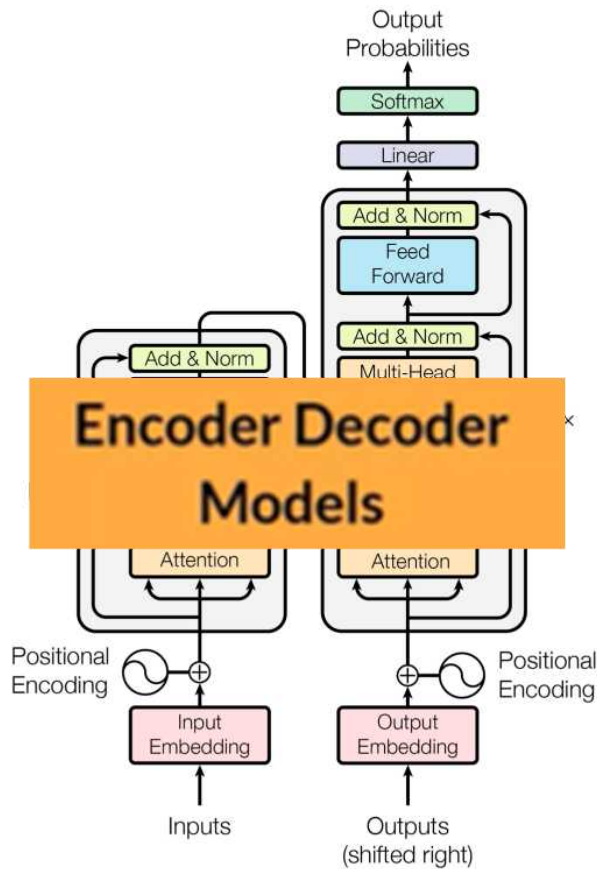
- 여러 개의 attention 을 병렬로 사용한 후 Attention Head 를 연결
→ 다른 시각으로 단어 간의 상관 관계 파악
- 각각이 random 하게 초기화 되므로 training 후 다른 subspace 표시



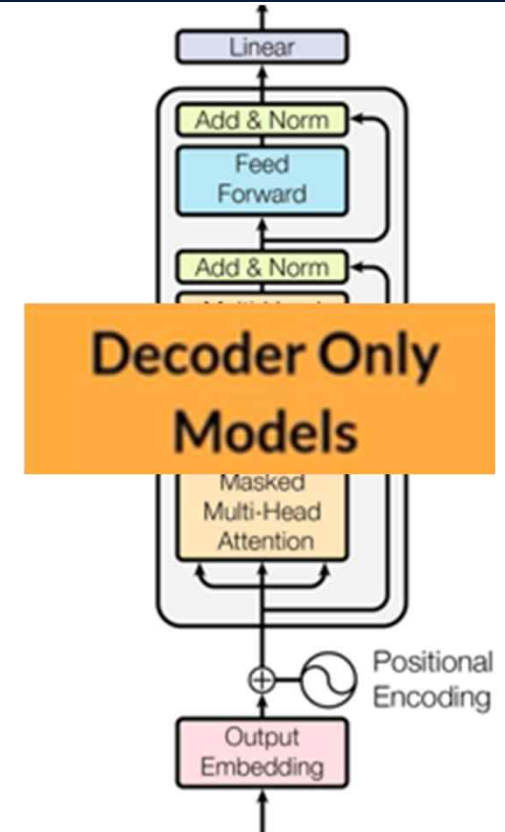
Transformers 파생 모델



- BERT



- T5
- BART



- GPT 모델 제품군
- BLOOM
- Jurassic
- LLaMA

GPT-3 의 탄생

- Transformer의 decoder 부분을 가능한 높이 쌓은 구성(96 layers)
 - 대용량 dataset(300 billion tokens of text)과 computing 자원을 이용하여 transformer 기반 language model 학습
 - language model – next word prediction model
 - GPT-3 training cost는 Tesla V100 cloud 사용 기준 1회 당 약 \$4.6M
 - fine-tuning 없이 사용하는 model
 - Parameter 수를 1.5 billion 에서 175 billion 으로 확장
 - GPT-3 가 생성한 가짜 기사 구분 by human – 52% (인간은 구분 못함)
-

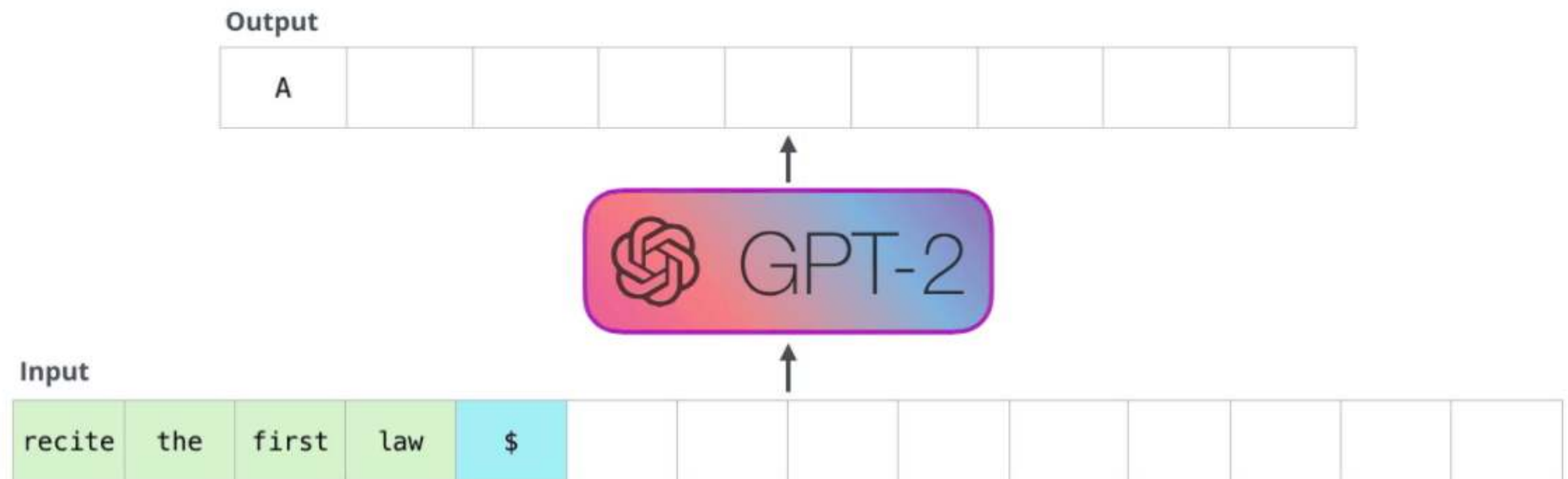
Zero / One / Few-shot Learning과 GPT-3

학습 방식	학습 데이터	Model이 하는 일
지도학습	말, 얼룩말, 젓소를 구분하기 위한 다량의 사진 필요	학습한 대상을 구분
Zero shot learning	말과 젓소의 사진만 학습 → "얼룩말은 젓소의 색을 가진 말 " 이라는 설명	얼룩말 사진을 구분
One shot learning	한 장의 원숭이 사진	다른 원숭이 사진 구분
Few shot learning	몇 장의 강아지 사진	다른 강아지 사진 구분

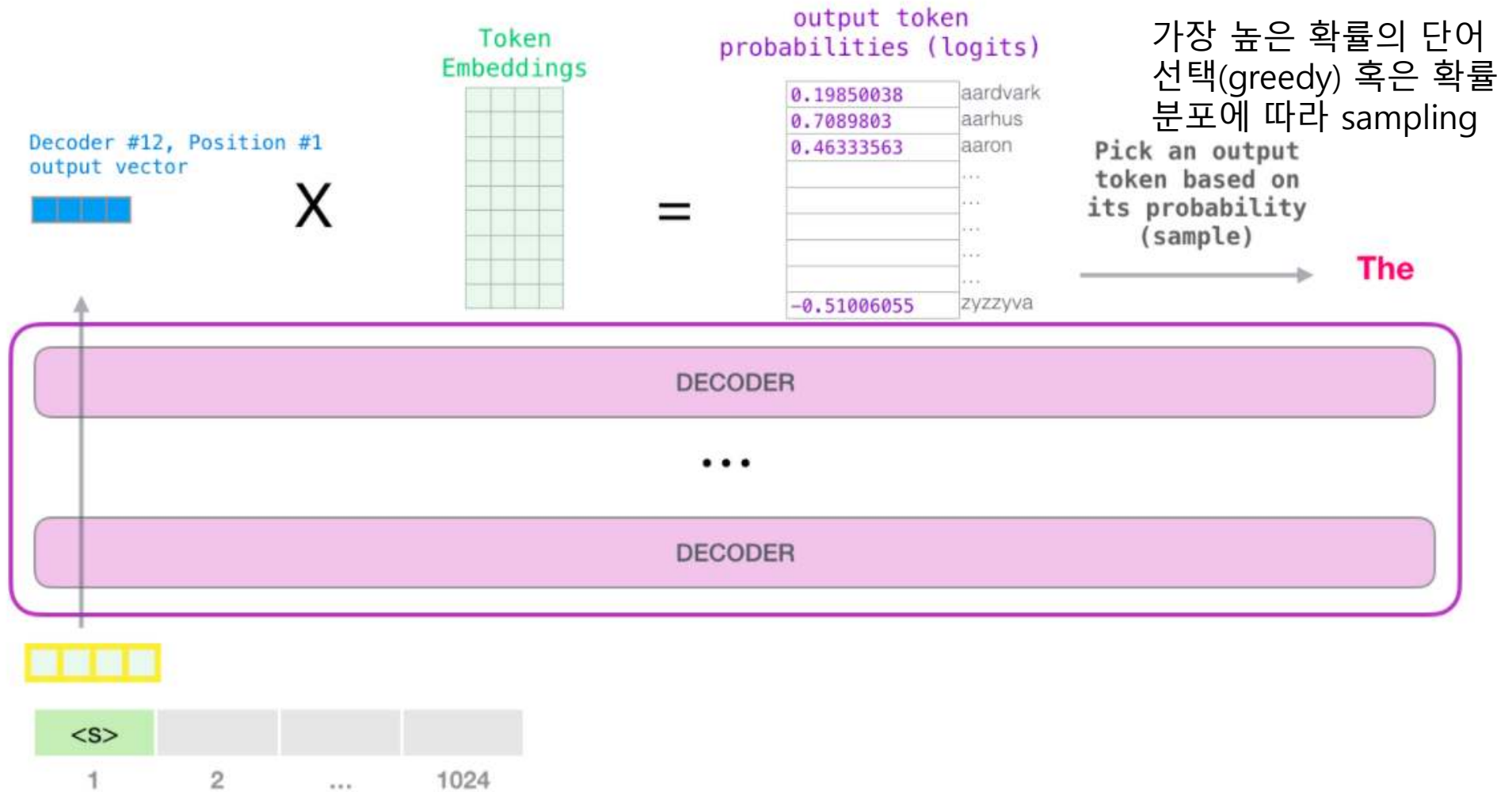
- GPT-3는 fine-tuning 없이 zero / one / few-shot 모두 가능하며, prompt의 예시가 많을수록 성능 향상 → "Few-Shot Learners"

GPT 는 auto-regressive 한 입력 구조

- 각 토큰이 생성 된 후 해당 토큰이 입력 시퀀스에 추가
- 그 새로운 시퀀스는 다음 단계에서 모델에 대한 입력이 된다



Output token probability



Decoding Strategy

- Greedy 전략
 - softmax 분포 중 가장 높은 확률 (argmax) 선택, 언제나 같은 결과
 - argmax : $P(\text{Over the line !} \mid \text{Çizgiyi geçtin !})$
- Sampling 전략 (사후확률분포)
 - 분포 확률에 따라 Random sampling, 매번 결과가 바뀔 수 있음
 - `np.random.choice(len(probs), p=probs)`
 - **GPT-3 / ChatGPT는 temperature와 top-p sampling 사용**

temperature / top_p — 출력 확률 분포 제어

- **temperature (0 ~ 2, 기본값 1)**

- softmax 확률 분포의 날카로움을 조절
- 낮을수록 결정적(greedy에 근접) → 일관되고 정확한 답변
- 높을수록 낮은 확률의 단어도 선택 → 다양하고 창의적인 답변

- **top_p (0 ~ 1, 기본값 1) — Nucleus Sampling**

- 누적 확률이 p에 도달할 때까지의 상위 토큰만 후보로 제한
- 예) top_p=0.1 → 상위 10% 확률 질량 내에서만 sampling

- **권장 사용법**

- 두 파라미터 중 하나만 조정 (동시 조정은 비권장)
 - 코드 생성·사실 질의: 0~0.2 / 일반 대화: 0.7 / 창작: 1.0 이상
 - 단, GPT-5 등 추론(reasoning) 모델은 사용자에게 temperature/top_p를 지원하지 않음
-

실습 : 150_autoregressive_language_generation

- Autoregressive 문장 생성
- Huggingface의 transformers library 사용
- Pre-trained GPT-2 모델 이용

```
1 # 문장 시작 부분
2 input_text = "Once upon a time"
3 input_ids = tokenizer.encode(input_text, return_tensors="pt")
4 input_ids
```

```
tensor([[7454, 2402, 257, 640]])
```

```
tensor([[7454, 2402, 257, 640, 11]])
tensor([[7454, 2402, 257, 640, 11, 612]])
tensor([[7454, 2402, 257, 640, 11, 612, 373]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257, 582]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257, 582, 508]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257, 582, 508, 5615]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257, 582, 508, 5615, 287]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257, 582, 508, 5615, 287,
257]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257, 582, 508, 5615, 287,
257, 7404]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257, 582, 508, 5615, 287,
257, 7404, 1444]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257, 582, 508, 5615, 287,
257, 7404, 1444, 509]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257, 582, 508,
5615, 287, 257, 7404, 1444, 509, 17716]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257, 582, 508,
5615, 287, 257, 7404, 1444, 509, 17716, 322]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257, 582, 508,
5615, 287, 257, 7404, 1444, 509, 17716, 322, 13]])
tensor([[7454, 2402, 257, 640, 11, 612, 373, 257, 582, 508,
5615, 287, 257, 7404, 1444, 509, 17716, 322, 13, 679]])
Once upon a time, there was a man who lived in a village called Krakow. He
```

What is ChatGPT ?

- ChatGPT라고 불리는 GPT-3.5는 GPT-3의 작지만 대화에 특화된 버전.
이 모델은 채팅의 맥락, 구조 및 어조를 더 깊이 이해하면서 자연스럽게 대화를 이어가는 유일한 목적으로 훈련. 따라서 채팅과 관련하여 범용 GPT-3에 비해 훨씬 더 정확하고 빠른 모델을 만든다.
 - GPT-5는 ChatGPT의 진화된 Multi-Modal 버전.
텍스트나 이미지 입력을 받아 텍스트를 출력하는 대규모 다중 모드 모델로, 이전 모델들보다 더 넓은 일반 지식과 진보된 추론 능력을 통해 더 정확하게 어려운 문제를 해결할 수 있다.
-

ChatGPT Fine Tuning 과정

RLHF(Reinforcement Learning from Human Feedback)

- 1단계 : **Supervised Fine Tuning(SFT) model** 작성
 - GPT-3를 초기 model로 사용
 - 인간이 의도하는 정책을 학습 시키기 위해 인간 labeler가 모델이 생성한 출력에 점수 부여
 - 이렇게 얻어진 데이터를 이용하여 지도학습 방법으로 Fine Tuning
 - 2단계 : **Reward Model** 작성(인간 선호도 모방)
 - 1단계에서 Fine Tuning된 SFT 모델이 동일 질문에 대해 여러 답변을 생성하게 한 후 인간 labeler가 ranking을 매긴 데이터셋 수집
 - 이 데이터셋을 이용하여 새로운 reward model 학습
 - 3단계 : **SFT 모델을 강화학습 (PPO) 으로 최적화**
 - 2단계에서 학습한 reward model의 점수를 보상으로 사용
 - PPO(Proximal Policy Optimization) 알고리즘으로 SFT 모델의 정책을 업데이트하여 인간 선호에 맞는 응답을 생성하도록 강화학습
-

언어 생성 모델의 환각(Hallucination) 현상

- "인공지능(AI) 챗봇에 발생하는 환각(할루시네이션) 현상은 여전한 AI 걸림돌입니다. 명확한 해결책도 없습니다. 사회에 AI를 안전하게 뿌리내리려면 환각 현상을 어떻게든 해결해야 합니다." – Sundar Pichai(구글 CEO)



문재인 대통령의 미국 방문 때 바이든 대통령과 몇차례의 회담을 했었나?



문재인 대통령은 2023년 5월 20일부터 22일까지 미국을 방문했습니다. 이 기간 동안 문재인 대통령은 조 바이든 미국 대통령과 총 2차례의 회담을 가졌습니다. 첫 번째 회담은 20일 오후 워싱턴 백악관에서 열렸으며, 두 번째 회담은 22일 오전 워싱턴 국무부에서 열렸습니다. 두 차례의 회담에서 양국 정상은 한미 동맹 강화, 북핵 문제, 경제 협력 등 다양한 분야에 대해 논의했습니다.



윤석열 대통령의 미국 방문 때 바이든 대통령과 몇차례의 회담을 했었나?

[다른 답안 보기](#) ▼



윤석열 대통령은 2023년 4월 20일부터 22일까지 미국을 방문했습니다. 이 기간 동안 윤석열 대통령은 조 바이든 미국 대통령과 두 차례의 회담을 했습니다. 첫 번째 회담은 4월 20일 백악관에서 열렸고, 두 번째 회담은 4월 21일 백악관에서 열렸습니다. 두 차례의 회담에서 윤석열 대통령과 조 바이든 대통령은 한미동맹 강화, 북한 핵 문제, 경제안보 등 다양한 분야에 대해 논의했습니다.

환각(Hallucination)의 원인과 완화 방법

• 왜 발생하는가?

- LLM은 '다음 단어 확률 예측' 모델 — 사실 검증 메커니즘이 없음
- 학습 데이터의 한계 (knowledge cutoff, 데이터에 포함된 오류)
- 유창함(fluency)을 사실성보다 우선하는 생성 방식

• 완화 방법

- 근거 문서를 검색해 프롬프트에 주입 (RAG) → File Search / Vector Store
 - 실시간 정보 연결 → Web Search, Function Calling, MCP
 - 출력 형식 제약 → Structured Outputs
 - temperature 낮추기, 프롬프트에 '모르면 모른다고 답하라' 지시
 - 추론(Reasoning) 모델 활용 — 내부 사고 과정으로 다단계 검증
 - **명확한 지시·맥락·예시로 프롬프트를 설계 → Prompt Engineering (다음 장에서 계속)**
-

Prompt Engineering

(Prompt Optimization)

Prompt Engineering 이란?

- 인공지능, 특히 자연어 처리를 하는 AI 모델에게 효과적인 질문이나 지시를 제공하는 기술과 방법

주요 요소	설명
명확성과 구체성	질문이나 지시가 명확하고 구체적이어야 AI가 원하는 답변을 더 쉽게 제공할 수 있음.
문맥 이해	충분한 배경 정보나 문맥을 제공하여 질문이나 요청 의도를 더 잘 이해하도록 함.
정확한 목적 전달	사용자가 특정 포맷, 스타일, 또는 정보의 종류를 원하는 경우, 이를 명시적으로 표현
반복과 조정	초기 프롬프트에 대한 AI의 반응을 분석하고, 필요한 경우 프롬프트를 조정하여 원하는 결과를 얻는 과정

Prompt Engineering 관련 용어

- Prompt – 모델에게 주는 가이드 (입력 텍스트)
 - Instruction (명령) – prompt의 핵심. 텍스트 요약, 번역, 분류 등
 - Context (문맥) – instruction을 구체화 할 수 있는 추가 정보
 - Input Data – AI 모델이 지시한 작업을 완료하기 위해 처리할 정보
 - Output Indicator – 원하는 출력 형식·종류 지정 (암시적 or 명시적)
→ 네 가지 구성 요소의 실제 예는 다음 슬라이드
 - 추론(Inference) – 모델이 prompt로부터 텍스트를 생성하는 행위
 - Completion – 출력 텍스트
 - Context window – 프롬프트에 사용할 수 있는 전체 텍스트 양 또는 메모리
-

Prompt Engineering 예

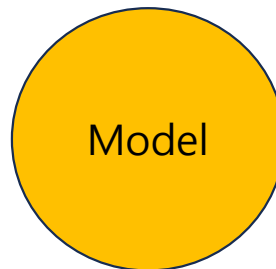
- **Instruction (명령):**
"다음 내용을 요약해 주세요."
 - **Context (문맥):**
"다음 내용은 민주주의와 공산주의에 대한 비교입니다. 중요한 사항들을 명확하게 이해할 수 있도록 주요 내용을 간략히 정리해야 합니다."
 - **Input Data:**
"공산주의는 생산수단의(중략) 제2차 세계대전의 경로를 통하여 동유럽 및 아시아의 공산주의권이 성립하기 시작했다. 그러나, 1991년 소비에트 연방이 붕괴한 이후부터 공산주의 운동은 사실상 쇠퇴의 일로를 맞고 있다"
 - **Output Indicator:**
"민주주의와 공산주의 비교의 핵심 포인트를 표로 정리해 주세요."
-

상황 내 학습(in-context learning) – zero shot inference

- 예시 없이 instruction만으로 답변 생성
- 모델의 사전 학습 지식에만 의존하는 가장 단순한 방식
- Guide가 없어 원하는 형식·내용의 답이 아닐 가능성 높음

Prompt

미래의 자동차가 어떤 모습
일지 상상해 보세요?

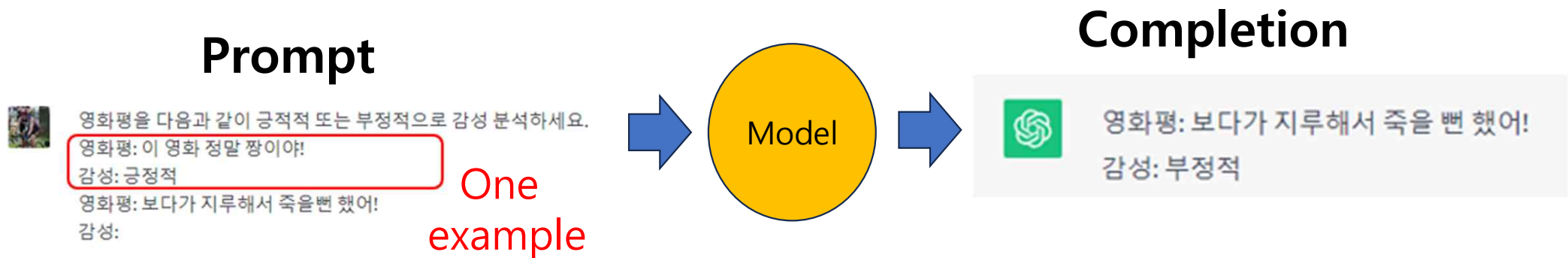


Completion

미래의 자동차는 현재의 기술
과 추세를 기반으로 상당히
진보된 모습을 가질 것입니다.
몇 가지 가능성은 다음과 같
습니다:
(이하 생략)

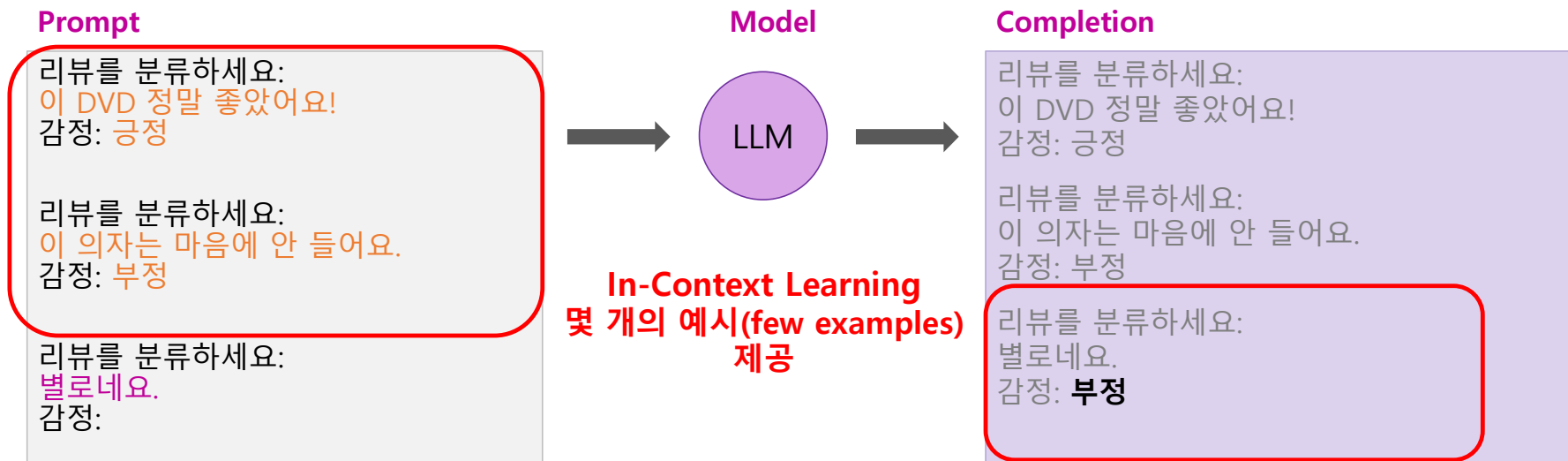
상황 내 학습(in-context learning) – one shot inference

- 원하는 작업의 예시 1개(example, shot)를 instruction과 함께 제시
- 모델은 제공된 예와 유사한 새로운 데이터를 생성 혹은 분류



상황 내 학습(in-context learning) – few shot inference

- 예시를 여러 개(shots) 제시하여 작업의 패턴을 더 명확하게 전달
- 예시가 많을수록 성능 향상 (zero < one < few) → "Few-Shot Learners"



COT(Chain-of-thought) Prompting

- LLM의 추론 능력 개선을 위해 Google 연구진이 제안 (<https://arxiv.org/pdf/2201.11903v1.pdf>)
 - 복잡한 문제를 중간 추론 단계(intermediate steps)로 분해하여 해결
 - 수행 단계
 1. 문제를 여러 단계로 분해
 2. 각 단계를 지시하는 프롬프트 작성
 3. 출력 결과를 다음 단계의 프롬프트에 입력
 4. 반복하여 최종 답 도출
 - 간단하게는 "step-by-step으로 풀어줘" 한 줄만으로도 추론 유도 가능 (zero-shot CoT)
 - 단, GPT-5 등 추론 모델은 내부적으로 CoT를 수행하므로 별도 지시 불필요 (역효과 가능)
-

Good Prompt 작성 – 원칙과 전략

작성 원칙

- Context – LLM 에게 context 작성을 맡기지 말고 우리가 제공
- Clear, non ambiguous task – LLM이 달성해야 할 구체적인 목표, 목적 설정
- Iteration – 출력 결과를 보고 prompt를 점진적으로 개선

세 가지 전략

- Zero-shot prompt : 예시 없는 prompt
- Few-shot prompt : 몇 개의 예시를 보여 주며 질문
- COT (Chain-of-thought) : Step-by-step 설명 요구
 - 같은 prompt로 여러 결과를 만들고 좋은 결과물을 선택 → 시간/비용 trade-off

GPT-5 등 추론 모델은 zero-shot 우선, 복잡한 출력 형식이 필요할 때만 few-shot

Core API Concepts

OpenAI API 주요 Component

주요 Component	Component 설명
Model	GPT-5, GPT-5-mini 등 다양한 언어 모델을 선택하여 텍스트 생성, 요약, 번역 등을 수행
Endpoint	responses, embeddings 등 API 요청을 처리하는 주요 경로
Prompt	모델에게 주는 입력 메시지. 역할(role)과 내용(content)으로 구성됨 (ex. developer, user, assistant)
Messages	대화형 모델에서 사용하는 대화 이력 구조. [{role: "user", content: "..."}, ...]
Token	모델이 이해하는 최소 단위(단어보다 작은 단위). 요청/응답의 길이 제한과 비용 산정 기준
Function Calling / Tool Use	함수 형태로 외부 작업(예: 계산, DB조회)을 수행할 수 있도록 모델이 함수를 호출
Embedding	텍스트를 고차원 벡터로 변환하여 검색, 유사도 분석 등에 사용
API Key	OpenAI API 사용을 위한 인증 키. 요청 시 Authorization: Bearer 헤더에 포함

Responses API

- OpenAI의 Chatting API를 사용하여 다음과 같은 응용 프로그램을 구축할 수 있다.
 - 문서 초안 작성
 - 컴퓨터 코드 작성
 - 지식 기반에 대한 질문에 답하기
 - 텍스트 분석
 - 소프트웨어에 자연어 인터페이스 제공
 - 다양한 과목에 대한 튜터링
 - 언어 번역
 - 게임용 캐릭터 시뮬레이션
-

OpenAI API role 의 종류

역할(role)	설명	예제
user	모델에게 특정 출력을 요청하는 메시지. ChatGPT를 사용할 때 사용자가 입력하는 일반적인 메시지와 동일함.	"프로그래밍에 대한 시를 한편 작성해줘."
developer	사용자 메시지보다 우선적으로 적용되는 모델 지침. 이전에는 "system prompt"라고 불렀음.	"당신은 유용한 프로그래밍 도우미입니다."
assistant	모델이 생성한 응답 메시지. 이전 대화에서 생성된 메시지를 포함할 수도 있음.	"물론이죠! 다음은 프로그래밍에 대한 짧은 시입니다: \n\n코드 위에 별이 뜨고, 오류는 사라지네..."

Token 관리

- "ChatGPT is great!" → 6개의 토큰으로 인코딩
["Chat", "G", "PT", " is", " great", "!"]

`{"role": "user", "content": "2020년 코리안 시리즈를 우승한 야구팀이 어디?"}`

➔ [882, [2366, 15, 75265, 226, 3396, 66391, 29102, 31495, 230, 45618, 29102, 96064, 230, 18918, 66822, 108, 18550, 117, 24486, 24814, 120, 89359, 169, 40934, 13094, 80402, 112, 90335, 30]

- 예를 들어, API 호출이 메시지 입력에서 10개의 토큰을 사용하고 메시지 출력에서 20개의 토큰을 받은 경우 30개의 토큰에 대한 요금이 청구됩니다
-

실습: 200_Text Generation and Prompts

기능	특징
상태 유지(Stateful)	✓ 서버 측에서 대화 상태를 관리(저장 및 이어쓰기 가능)
내장 도구 지원(Built-in Tools)	✓ <code>web_search</code> , <code>file_search</code> , <code>computer_use_preview</code> 등 자동 호출 가능
구조 & 사용 방식	<code>instructions</code> , <code>input</code> , <code>tools</code> , <code>store</code> , <code>previous_response_id</code> 기반 API
응답 형식 (Output)	<code>response.output_text</code> , <code>response.output</code> 구조로 단일 메시지 응답 및 도구 호출 결과 포함
사용 목적	✓ 에이전트/도구 사용, 파일/웹 검색, 상태 관리가 필요한 복잡한 워크플로우에 적합
미래 확장성	🔧 OpenAI Roadmap 중심 - Chat API의 기능을 통합 발전시키는 방향

생성형 Image Model

주요 Image 생성 모델

- 2014 GAN (Generative Adversarial Network)
 - 2021 CLIP : OpenAI. Image → Text 변환
 - 2021.12 GLIDE : OpenAI. Text → Image 변환
 - 2022.4 DALL-E 2 : OpenAI. Text → Image 변환
 - 2022.6 Stable Diffusion : Open Source로 공개
 - 2022.7 Midjourney : 예술적 스타일로 대중적 인기
 - 2025.11 FLUX.2 / Imagen 4 / Midjourney v7
: 프로덕션급 품질, 텍스트 렌더링 및 캐릭터 일관성 대폭 개선
 - 2026.2 Nano Banana 2 : Google. Gemini 3.1 기반 고속 이미지 생성
 - 2026.4 GPT Image 2 : OpenAI. 추론 내장, 텍스트 정확도 대폭 향상
-

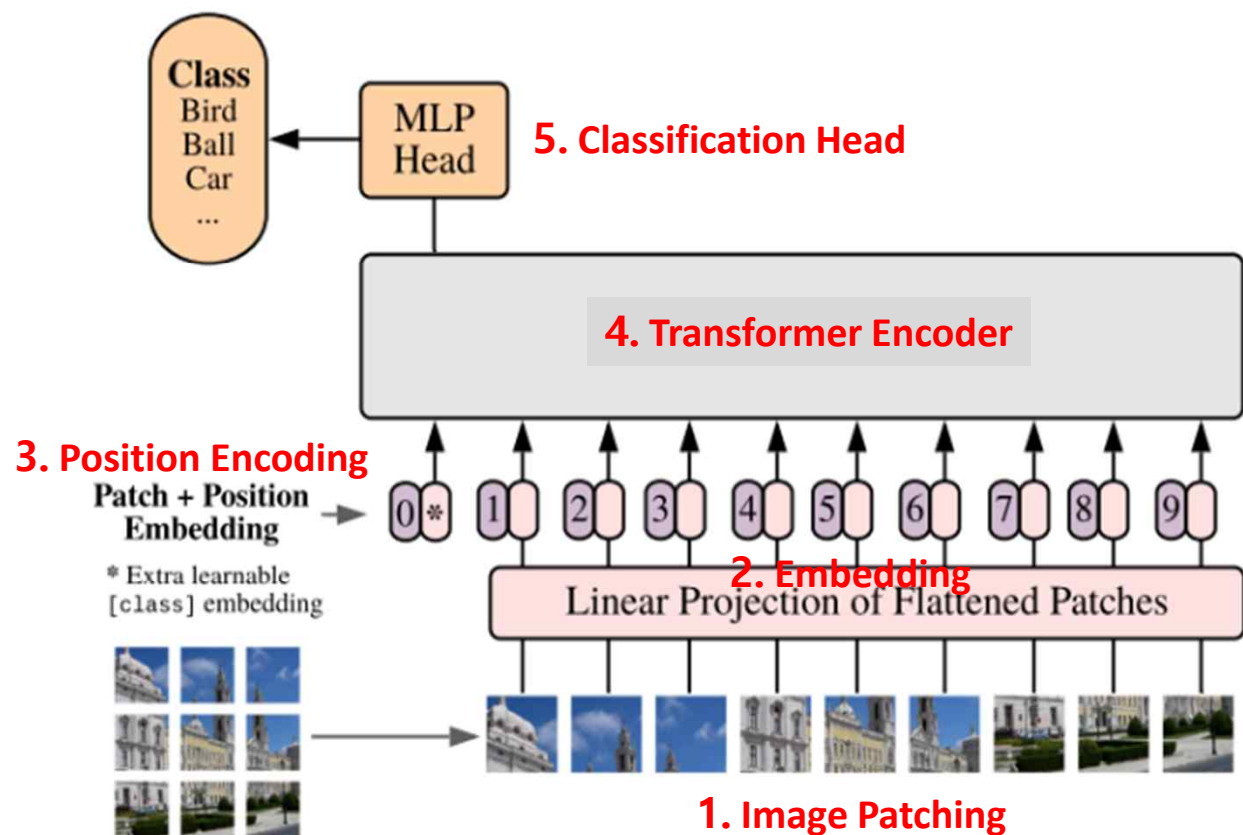
Vision Transformer

Vision Transformer ?

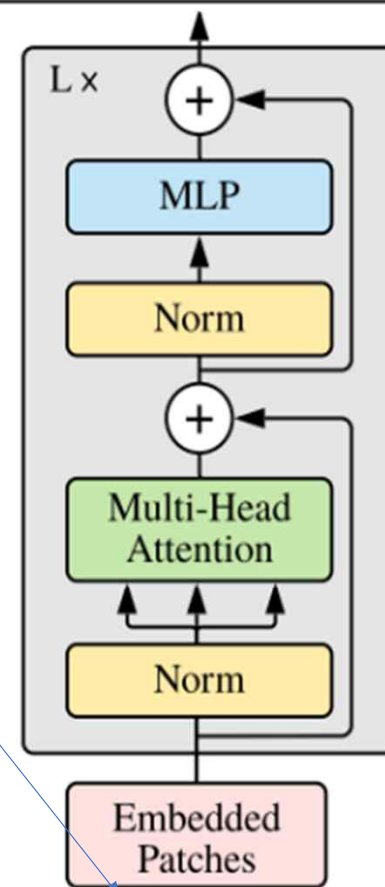
- CNN(Convolutional Neural Networks)이 주로 사용되었던 이미지 처리에 NLP에서 큰 성과를 보이던 Transformer를 적용
 - CNN이 가지고 있던 고정된 수용 영역의 한계를 극복하며, 전체 이미지를 고려한 글로벌한 정보를 활용
 - 많은 데이터와 큰 모델 규모가 필요하다는 단점이 있지만, 데이터 증강 및 다양한 사전 훈련 전략을 통해 이러한 문제 해결
-

Model Overview

Vision Transformer (ViT)



Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M



ViT의 작동 방식

- 1. Image Patching:** 주어진 이미지를 여러 개의 고정 크기의 패치(patch)로 나눕니다.
예를 들어, 224×224 크기의 이미지를 16×16 크기의 패치로 나누면 총 196개의 패치가 생깁니다.
 - 2. Embedding:** 각 패치를 1D 벡터로 평탄화(flatten)한 후, 선형 임베딩(linear embedding) 레이어를 통과시켜 고정된 크기의 임베딩 벡터로 변환
 - 3. Position Encoding:** 각 패치의 위치 정보를 주입하기 위해 위치 인코딩(position encoding)을 추가. 이는 NLP에서의 Transformer와 동일한 개념.
 - 4. Transformer Encoder:** 앞서 생성한 임베딩 벡터들은 Transformer 인코더에 입력됩니다.
Transformer 인코더는 여러 개의 attention 및 feed-forward 네트워크 레이어로 구성.
 - 5. Classification Head:** Transformer의 최종 출력은 분류 작업을 위한 MLP 레이어(또는 다른 헤더)에 입력
-

CLIP

(Contrastive Language-Image Pre-training)

Learning Transferable Visual Models From Natural Language Supervision

사용 dataset

- Image/Text pair dataset
 - MS-COCO, Visual Genome: 고품질/소규모 데이터 (100,000개)
 - YFCC100M (100M) → filtering 후 고품질 15M개 사용
 - 인터넷에 공개되어 있는 400M개의 (image, text) pair로 새로운 dataset 구성
 - 텍스트 query는 위키피디아에서 100번 이상 등장하는 단어 50만개로 구축
 - 데이터 균형을 위해 query당 이미지-텍스트 pair 개수를 최대 2만개로 제한
→ 같은 이미지를 설명하는 다양한 표현 생성
 - 위 3가지 데이터로 contrastive learning (대조학습) 수행
- * constrasive learning 은 서로 다른 두 데이터 포인트가 얼마나 다른지 (또는 유사한지) 를 비교하거나 측정하여 유사한 데이터 포인트들이 가까이 있도록 임베딩 공간을 구성
-

Data example



a black dog with white spots



Dreamstime.com

Black Dog with White Spots...



Quora

What are some dog breeds that ar...



Daily Mail

Rowdy the Labrador's viti...



Can Stock Photo

Puppy black and white with spots. C...



Adobe Stock

Black dog with white spots on the legs, chest ...



PetHelpful

90+ Unique Names for Dogs With Spot...



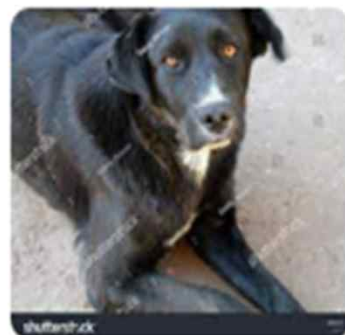
Alamy

Black dog with white spots on the legs...



Reddit

Black Dalmatian With White S...



Shutterstock

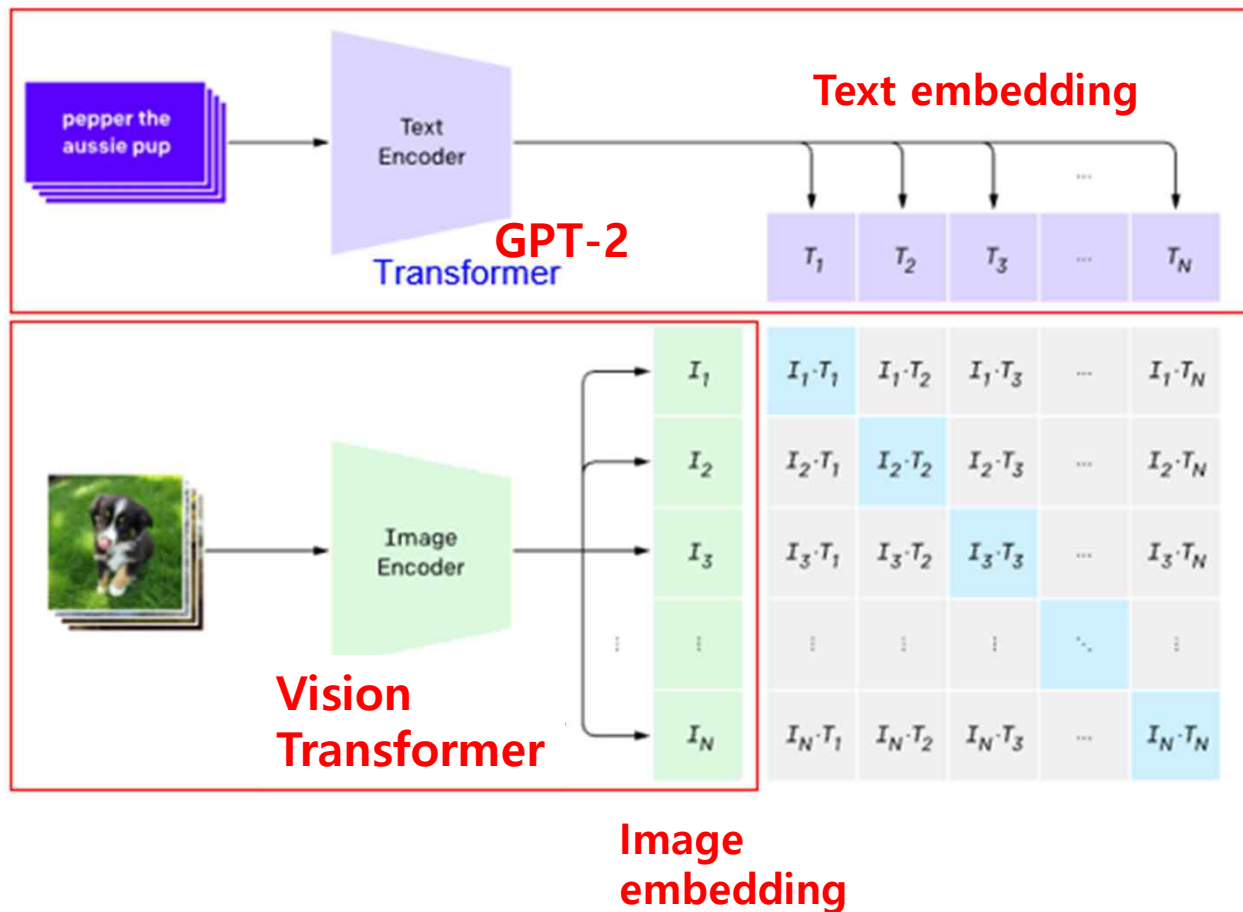
Black Dog White Spots Looks Yell...



Reddit

My all-black dog develope...

Contrastive Pre-Training (대조 학습)



- N개의 이미지, 텍스트를 각각 encoder를 통과시켜 embedding vector 계산 (N: batch 사이즈)
- NxN matrix의 대각선 부분이 positive pair, 그 외 부분은 모두 negative pair (N개의 positive pair, $N^2 - N$ 개의 negative pair)
- Positive pair의 cosine 유사도는 maximize 하고, negative pair의 cosine 유사도는 minimize 하도록 train

실습: 160_Interacting_with_CLIP

- CLIP model을 download 받아 실행
- 임의의 이미지와 텍스트 간의 유사성 계산
- Zero shot 이미지 분류 수행 방법 이해

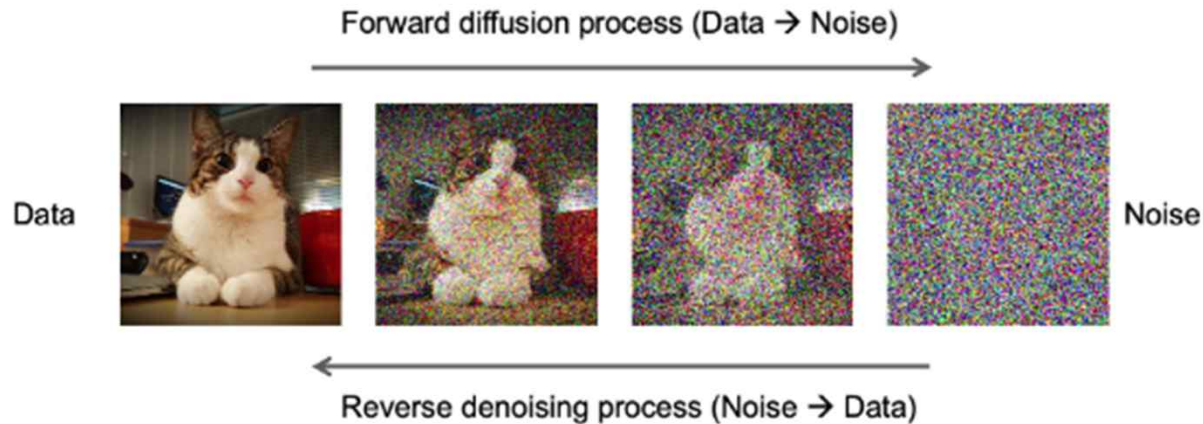


	camera.png	rocket.jpg	chelsea.png	coffee.png	astronaut.png	page.png	motorcycle_right.png	horse.png
a person looking at a camera on a tripod	0.30	0.21	0.21	0.14	0.19	0.19	0.16	0.20
a rocket standing on a launchpad	0.20	0.30	0.18	0.14	0.19	0.17	0.16	0.17
a facial photo of a tabby cat	0.16	0.12	0.31	0.17	0.12	0.12	0.12	0.15
a cup of coffee on a saucer	0.17	0.12	0.18	0.29	0.15	0.14	0.12	0.15
a portrait of an astronaut with the American flag	0.17	0.22	0.17	0.15	0.28	0.13	0.15	0.16
a page of text about segmentation	0.20	0.16	0.20	0.20	0.15	0.35	0.16	0.20
a red motorcycle standing in a garage	0.12	0.16	0.15	0.13	0.15	0.14	0.32	0.16
a black-and-white silhouette of a horse	0.21	0.15	0.15	0.15	0.11	0.17	0.17	0.35

Diffusion Model

Diffusion Model (확산 모델) ?

- Diffusion은 Noise가 퍼져 나간다는 의미
- 점진적인 노이즈 프로세스를 반대로 하여 데이터를 생성하는 방법을 학습
- GAN 수준의 High Quality + GAN보다 뛰어난 Diversity 구현
- 이미지에 노이즈를 점진적으로 추가(Forward / Diffusion Process)하여 순수한 가우시안 노이즈로 만든다
- 이 체인을 거꾸로 따라가며 점진적으로 노이즈를 제거(Denoising / Reverse Process)하여 이미지를 생성

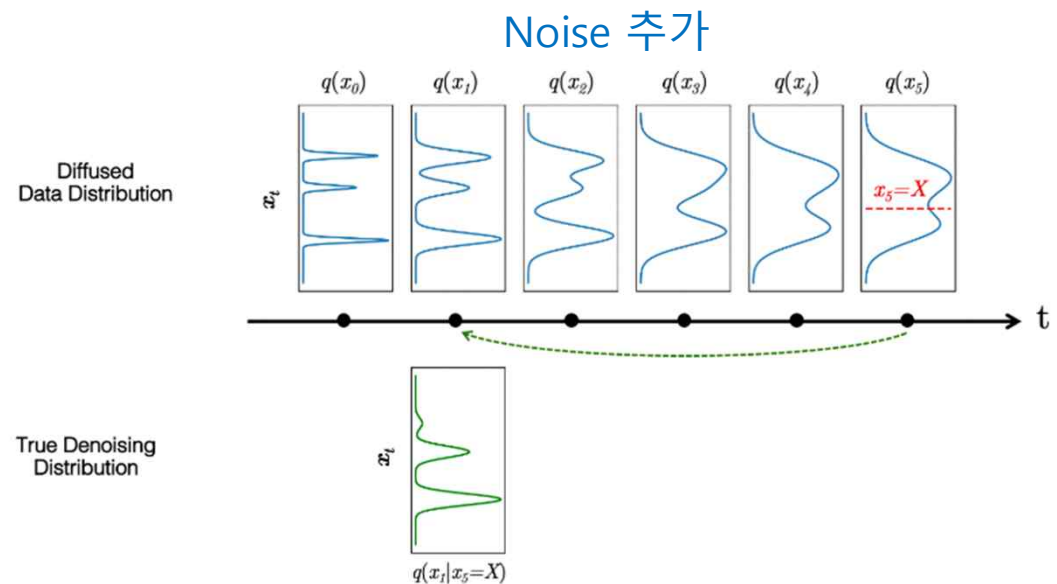


확산 모델의 기본 Idea

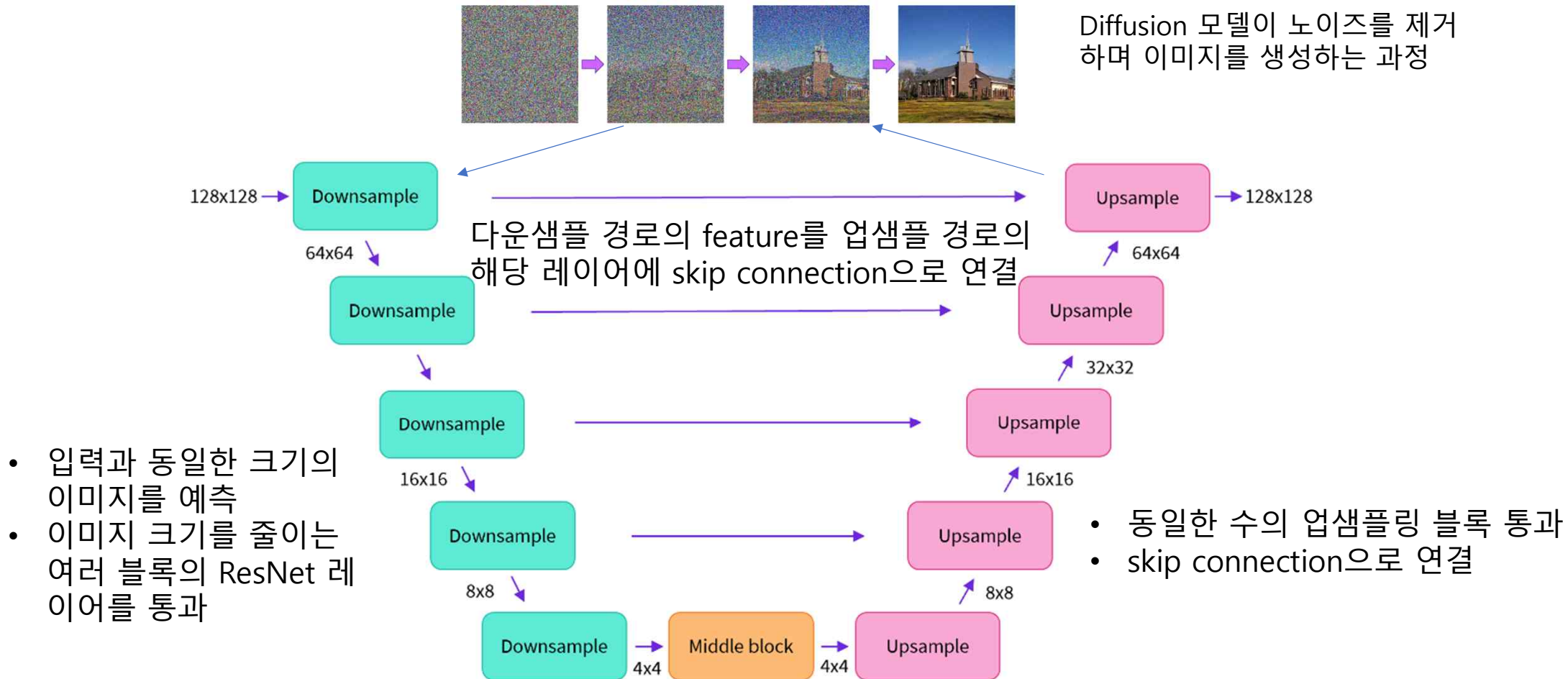
- 짧은 시간 내에서는 이미지에 노이즈를 더할 때나 없앨 때나 모두 gaussian 분포 내에서 결정된다는 것이 핵심 아이디어.
- 정규분포는 평균과 분산만 알면 분포를 예측할 수 있는데 Diffusion model 이 하는 일은 역으로 돌아갈 때 평균과 분산을 맞추는 일을 한다.
- 짧은 단계 (ex. 1000 step)로 나누어 단계별의 평균과 분산을 AI가 계산 (분산은 대개 1로 고정)

[그림 해설]

- 위: $q(x_0) \rightarrow q(x_5)$ 로 갈수록 노이즈가 더해져 가우시안 분포로 퍼짐 (Forward)
- 녹색 점선: 거꾸로 돌아가는 Reverse(Denoising) 방향
- $q(x_1|x_5=X)$: 노이즈 X 가 주어졌을 때 한 단계 이전 분포를 추정 → 모델의 학습 목표

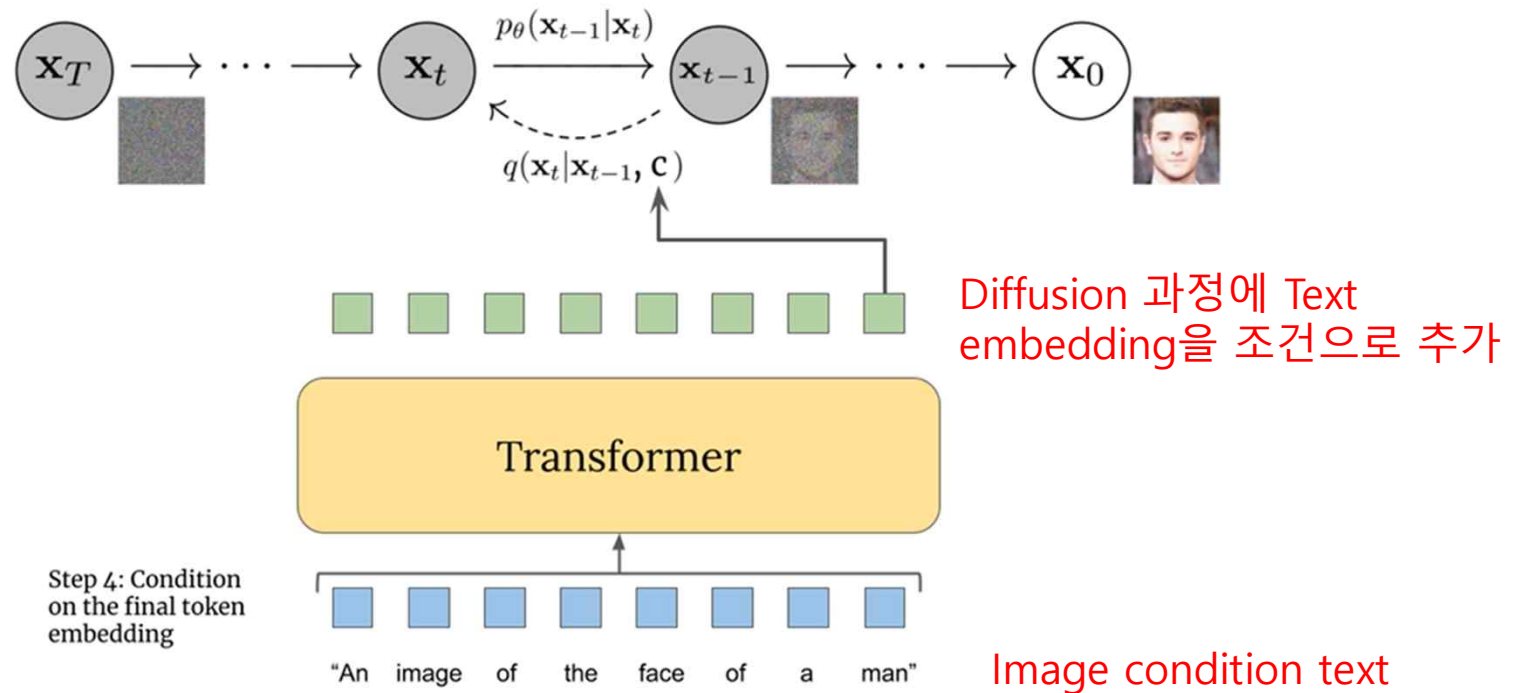


Diffusion Model 구조 – UNet 사용



OpenAI GLIDE (Generative Language-Image Diffusion Estimator)

- 텍스트 설명을 입력하면 해당 이미지를 점차적으로 생성해내는 **확산 기반 이미지 생성 모델**





DALL-E/GPT-Image-1

Image Generation AI

- What is DALL-E ?
 - 2020. 9.28에 OpenAI에서 발표. 120 억개의 매개변수로 이루어진 Transformer 기반의 GPT-3 확장 형태. 텍스트 → 고품질 이미지 생성 모델
 - 동물과 물체의 의인화된 버전 생성, 관련 없는 개념을 그럴듯한 방식으로 결합, 텍스트 렌더링, 기존 이미지에 변형 적용 등 다양한 기능
 - 기존에 존재 하는 이미지가 아닌 완전히 새로운 이미지를 생성
 - 2021.10 DALL-E2 : DALL-E의 개선된 version <https://openai.com/dall-e-2/>
 - 2023. 9 DALL-E3 : 완전한 Multi-Modal model <https://openai.com/dall-e-3>
-

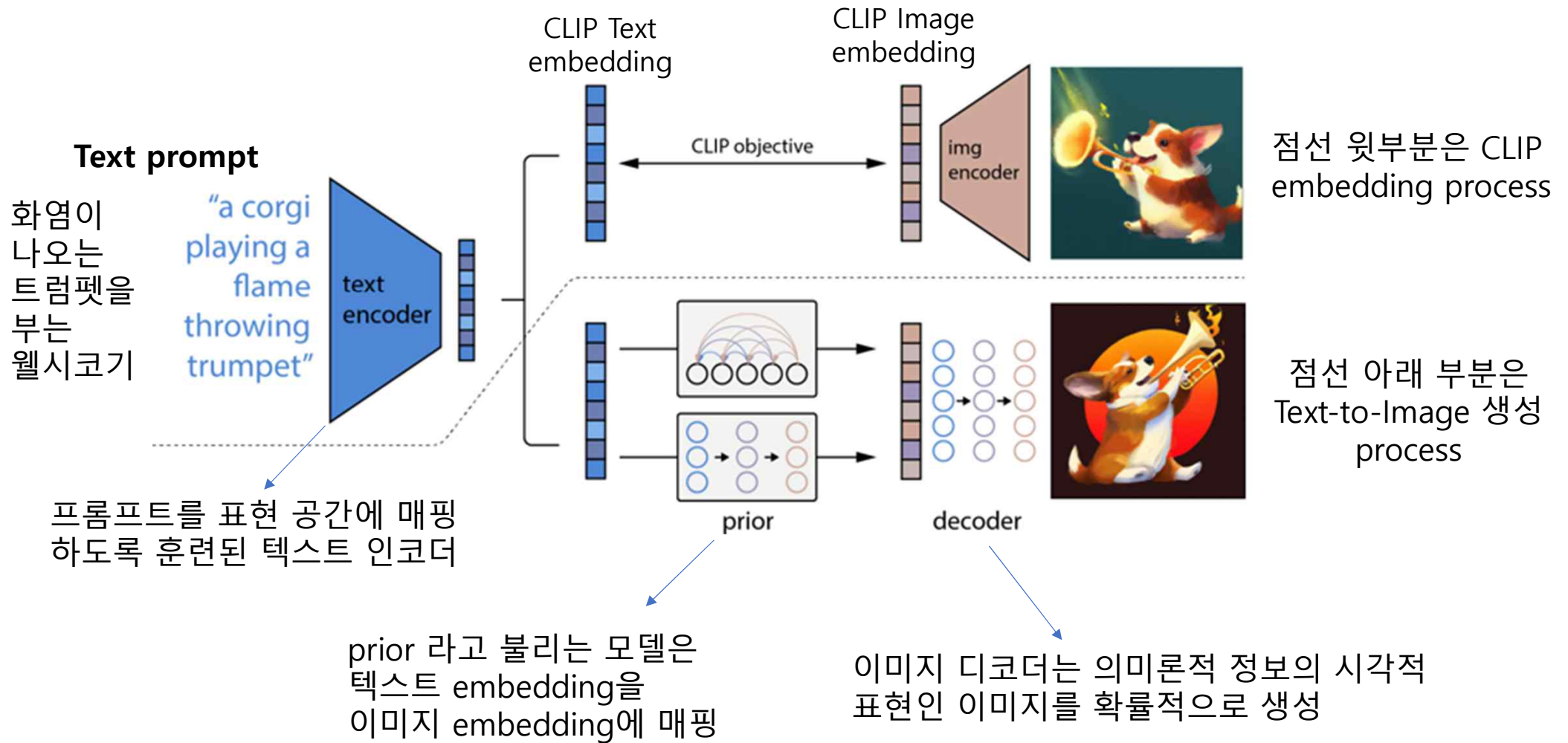
DALL-E3



Larry의 집을 그려줘 ➡



DALL-E2 Structure



DALL-E 3 vs GPT-Image-2 구조 비교

항목	DALL-E 3	GPT-Image-1
기본 아키텍처	Diffusion Model + CLIP	Autoregressive (GPT-4o 기반)
텍스트 이해	CLIP Text Encoder (별도)	GPT-4o LLM이 직접 이해
생성 방식	노이즈 → 점진적 제거	픽셀을 순서대로 하나씩 예측
텍스트 렌더링	낮음 (~60%)	매우 높음 (~98%)
논문 (Architecture)	공개	비공개

OpenAI 이미지 생성: Image API vs Responses API

구분	Image API	Responses API
기본 개념	Generations / Edits 전용 엔드포인트	대화 플로우 안에서 built-in tool로 생성
모델 지정	GPT Image 모델(gpt-image-2) 직접 지정	메인라인 모델(gpt-5.5 등)이 tool로 처리
멀티턴 편집	❌ 단발성 (1 프롬프트 → 1 결과)	✅ previous_response_id로 반복 편집
입력 방식	이미지 바이트 (파일 업로드)	URL · Base64 · file ID 모두 지원
생성/편집 제어	엔드포인트로 구분 (generations/edits)	action 파라미터 (auto · generate · edit)
비용 구조	이미지 생성 비용만	이미지 비용 + 메인라인 모델 토큰 과금
권장 사용처	단일 프롬프트 생성·편집	대화형으로 반복 수정하는 경험

핵심: 이미지 자체가 목적이면 Image API, 대화 속에서 이미지를 다루는 것이 목적이면 Responses API

이미지 생성하기

OpenAI API를 사용하면 GPT Image 또는 DALL·E 모델을 이용하여 텍스트 프롬프트에서 이미지를 생성하고 편집할 수 있습니다. 이미지 생성 기능은 두 가지 API를 통해 이용할 수 있습니다.

- image API : gpt-image-1.5, gpt-image-1, 및 gpt-image-1-mini, dall-e-3 등. 단순 이미지 생성
- responses API :
 - 다중 턴 편집 : 프롬프트를 통해 이미지를 반복적으로 고품질로 편집합니다.
 - 유연한 입력 : 바이트 데이터뿐 아니라 이미지 파일 ID도 입력 이미지로 허용합니다.

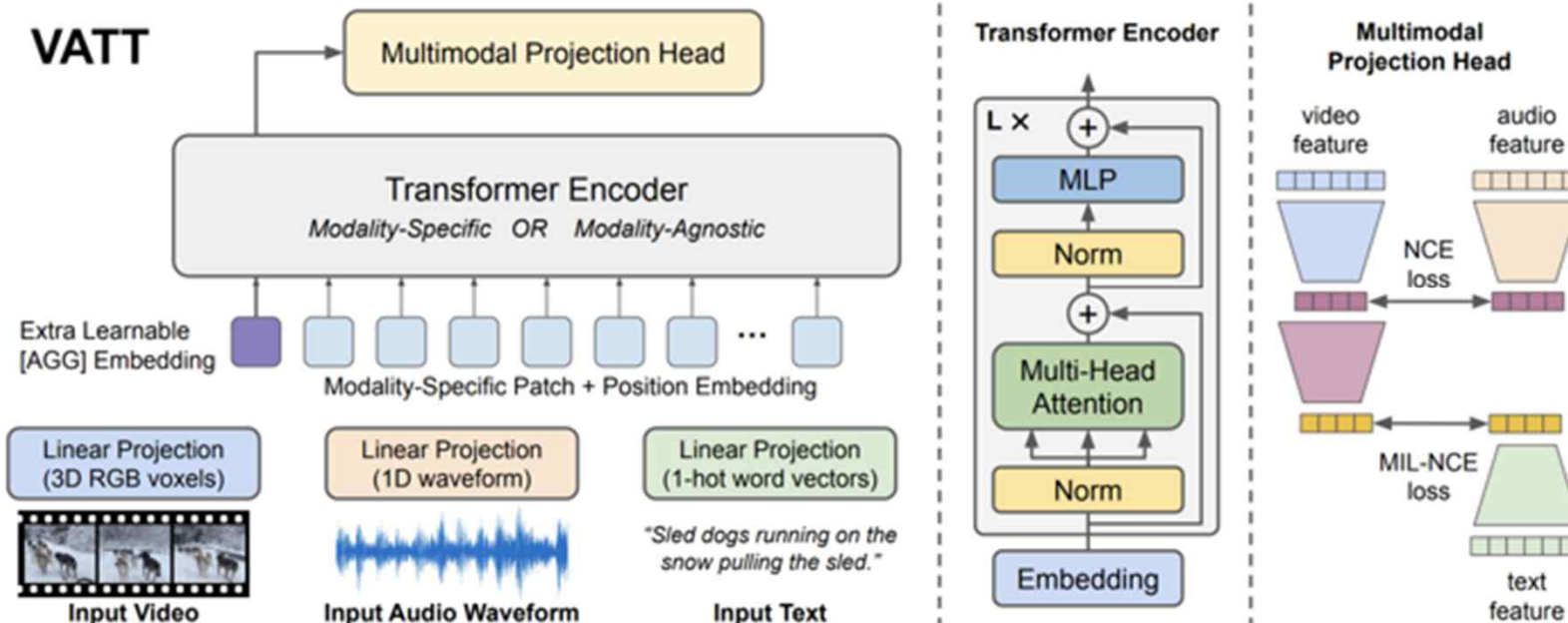
What is Multimodal AI ?

- 멀티모달 AI(Multimodal AI)는 다양한 유형의 입력 데이터(예: 텍스트, 이미지, 오디오, 비디오 등)를 처리할 수 있는 인공지능 기술
- 서로 다른 형태의 데이터를 통합하여 처리하고, 이를 기반으로 의사결정을 내리거나 예측을 수행하는 데 사용
- 인간이 정보를 처리하고 이해하는 방식이 다양한 감각을 통합하여 이루어진다는 점에서 착안

예) 이미지 내의 객체를 인식하는 동시에 이미지와 관련된 텍스트 설명을 이해하고, 두 데이터 소스의 정보를 통합하여 더 정확한 분석 수행

VATT(Vision, Audio, Text Transformer) 모델

초기 멀티모달 Transformer 연구 사례 (2021, Google)



Transformer Encoder를 통해 얻어진 특징 벡터들은 공통 공간으로 투영하여 대조학습을 시킵니다.

※ GPT-4o-GPT-5 등 현재 첨단 모델의 내부 구조는 비공개이며, VATT와 달리 모든 입력을 토큰으로 통합 처리하는 네이티브 멀티모달 방식으로 알려져 있음.

Input Video: 3D RGB Voxel로 변환된 비디오 프레임. 각 비디오 프레임은 3차원 형태로 입력됩니다.

Input Audio Waveform: 1D 파형으로 변환된 오디오 데이터. 오디오 입력은 파형 형태로 처리됩니다.

Input Text: 1-hot word vectors로 변환된 텍스트 데이터. 텍스트 입력은 단어 벡터 형태로 처리됩니다.

→ 각 모달리티의 입력 데이터는 선형 투영(Linear Projection)을 통해 임베딩 벡터로 변환

Vision API

- GPT-5, GPT-4o는 이미지 이해 능력 보유
 - GPT-5와 같은 multi-modal model 은 Text + 이미지를 입력으로 받아 그에 대한 질문에 답할 수 있습니다.
 - 모델은 이미지에 무엇이 있는지에 대한 일반적인 질문에 가장 잘 대답합니다.
 - 이미지에 있는 개체 간의 관계를 이해하지만, 이미지에 있는 특정 개체의 위치에 대한 자세한 질문에 답하기에는 아직 최적화되어 있지 않습니다. 예를 들어, 자동차의 색깔이 무엇인지, 냉장고에 있는 음식을 바탕으로 저녁 식사에 대한 아이디어가 무엇인지 물어볼 수 있지만, 방의 이미지를 보여주고 의자가 어디에 있는지 묻는다면 정확하게 대답하지 않을 수 있습니다.
-

Vision model의 한계점

- 의료 이미지: CT 스캔과 같은 의료 이미지를 해석하는 데 적합하지 않습니다.
 - 비영어: 일본어, 한국어 등 라틴 알파벳이 아닌 텍스트 이미지를 잘 처리하지 못합니다.
 - 작은 텍스트: 가독성을 높이려면 이미지 내 텍스트를 확대하세요.
 - 회전: 모델이 회전되거나 거꾸로 된 텍스트나 이미지를 잘못 해석할 수 있습니다.
 - 시각적 요소: 모델은 실선, 점선과 같은 색상이나 스타일이 다양한 그래프나 텍스트를 이해하는 데 어려움을 겪을 수 있습니다.
 - 공간 추론: 체스 위치 식별과 같이 공간 위치 파악 작업 수행에 어려움을 겪습니다.
 - 이미지 모양: 모델은 파노라마 및 어안 이미지로 인해 어려움을 겪습니다.
 - 계산: 이미지에 있는 개체의 정확한 개수가 아닌 **대략적인 개수만** 제공할 수 있습니다.
 - 안전상의 이유로 CAPTCHA 제출을 차단하는 시스템을 구현했습니다.
-

실습: 320_Vision 기능

Vision api

최신 언어 모델은 이미지 입력을 처리하고 분석할 수 있는데, 이를 '비전' 이라고 합니다 . '비전'을 통해 시각적 입력을 분석하고 이미지를 생성할 수 있습니다.

API	지원되는 사용 사례
Responses API	이미지를 분석하여 입력으로 사용하거나, 이미지를 출력으로 생성할 수 있음

이미지 분석

Vision은 모델이 이미지를 "보고" 이해하는 능력입니다. 이미지에 텍스트가 있으면 모델은 텍스트도 이해할 수 있습니다. 모델은 몇 가지 제약이 있더라도 물체, 모양, 색상, 질감을 포함한 대부분의 시각적 요소를 이해할 수 있습니다.

음성 AI — STT · TTS · Realtime API

- 오디오도 토큰이다

음성 파형을 오디오 토큰으로 변환하여 Transformer로 처리 — 텍스트 생성과 동일한 원리

- STT / TTS

STT: 음성 → 텍스트 (받아쓰기·자막·회의록)

TTS: 텍스트 → 음성 — '말하는 방식'(어조·감정)까지 프롬프트로 지시 가능

- 모델 발전

2022 Whisper → 2025 gpt-4o-transcribe · mini-tts
→ 2026.5 GPT-Realtime-2

기존

STT → LLM → TTS 파이프라인

단계마다 지연 누적, 어조·감정 정보 소실

단일 모델

Realtime API — Speech-to-Speech

음성을 직접 이해·생성 — 낮은 지연, 끼어들기 처리

시리즈

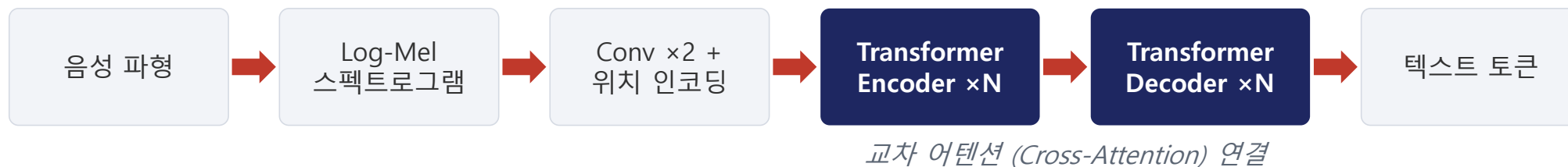
Realtime-2 · Translate · Whisper

음성 에이전트 · 동시통역(70+) · 저지연 STT — 분당 ~\$0.017

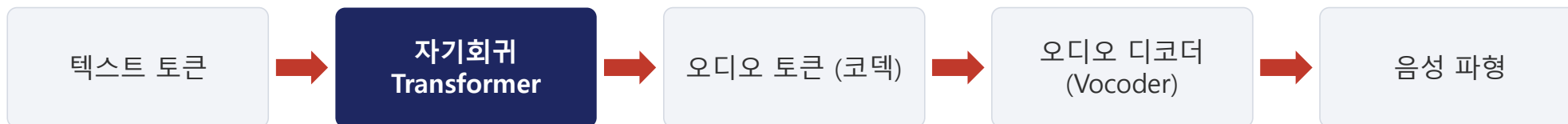
OpenAI Audio API: audio.transcriptions(STT) · audio.speech(TTS) · Realtime(WebSocket/WebRTC) — 음성 비서, 콜센터 상담, 실시간 통역에 활용

STT · TTS 모델 아키텍처

STT — Whisper · Encoder-Decoder Transformer (아키텍처·가중치 공개, 오픈소스)



TTS — 토큰 기반 생성 구조 (공개 설계 기준 · 상용 모델 내부는 비공개)



공통 원리: 모든 입력·출력을 토큰 시퀀스로 변환해 Transformer가 처리
STT는 오디오 토큰 → 텍스트 토큰, TTS는 텍스트 토큰 → 오디오 토큰

실습: 330_Audio_STT_TTS

음성 AI — STT · TTS · Realtime API

오디오도 토큰이다. 음성 파형을 오디오 토큰으로 변환하면 Transformer가 텍스트와 동일한 원리로 처리할 수 있습니다. 이 노트북에서는 OpenAI Audio API의 세 가지 핵심 기능을 실습합니다.

기능	엔드포인트	설명
TTS (음성 합성)	<code>audio.speech</code>	텍스트 → 음성. '말하는 방식'(어조·감정)까지 프롬프트로 지시
STT (음성 인식)	<code>audio.transcriptions</code>	음성 → 텍스트 (발아쓰기·자막·회의록)
Realtime (실시간)	<code>Realtime</code> (WebSocket/WebRTC)	음성을 직접 이해·생성. 낮은 지연, 끼어들기 처리

기존 파이프라인 vs 단일 모델

- 기존: STT -> LLM -> TTS — 단계마다 지연이 누적되고, 어조·감정 정보가 소실됨
- 단일 모델 (Realtime): 음성을 직접 이해·생성 — 낮은 지연, 끼어들기(barge-in) 처리

모델 발전

2022 Whisper -> 2025 gpt-4o-transcribe / gpt-4o-mini-tts -> 2026 GPT-Realtime-2

활용처: 음성 비서, 콜센터 상담, 실시간 통역(70+ 언어)

비디오 생성 AI

작동 원리 — Diffusion Transformer (DiT)

- 영상을 시공간 패치(spacetime patch)로 분할 — ViT의 이미지 패치를 시간 축으로 확장
- 노이즈 영상에서 점진적 제거(Diffusion)로 영상 생성, Transformer가 패치 간 관계 학습
- 최신 모델은 대사·효과음까지 영상과 동기화하여 동시 생성

- 2024.2 ● **OpenAI Sora 발표** 텍스트→영상 데모로 DiT 방식 대중화
- 2025.10 ● **Sora 2 / Veo 3 경쟁** 물리 법칙 준수, 오디오 동기 생성, 16~20초 영상
- 2026 ● **시장 재편** Sora 단계적 종료(앱 4월·API 9월 예정), Veo 3.1 · Kling 3.0 · Runway Gen-4.5 경쟁 가속

시사점: 비디오는 생성형 AI 중 모델 교체 주기가 가장 빠른 분야 — 특정 모델보다 'DiT + 시공간 패치' 원리를 이해하는 것이 중요

구조화된 출력

(Structured Output)

구조화된 출력 (Structured Outputs)

- JSON 스키마를 준수하는 응답을 보장
 - 구조화된 출력의 주요 이점
 - 잘못된 형식의 응답을 수동으로 검증하거나 재시도할 필요 없음.
 - 모델의 거부 응답을 프로그래밍 방식으로 감지 가능.
 - 응답 형식을 일관되게 유지하기 위해 강한 문장을 사용할 필요 없음.
 - 구조화된 출력은 두 가지 방식으로 사용할 수 있습니다.
 - 응답 형식(Response Format)
 - 함수 호출(Function Calling)
-

구조화된 응답 형식 (Response Format)

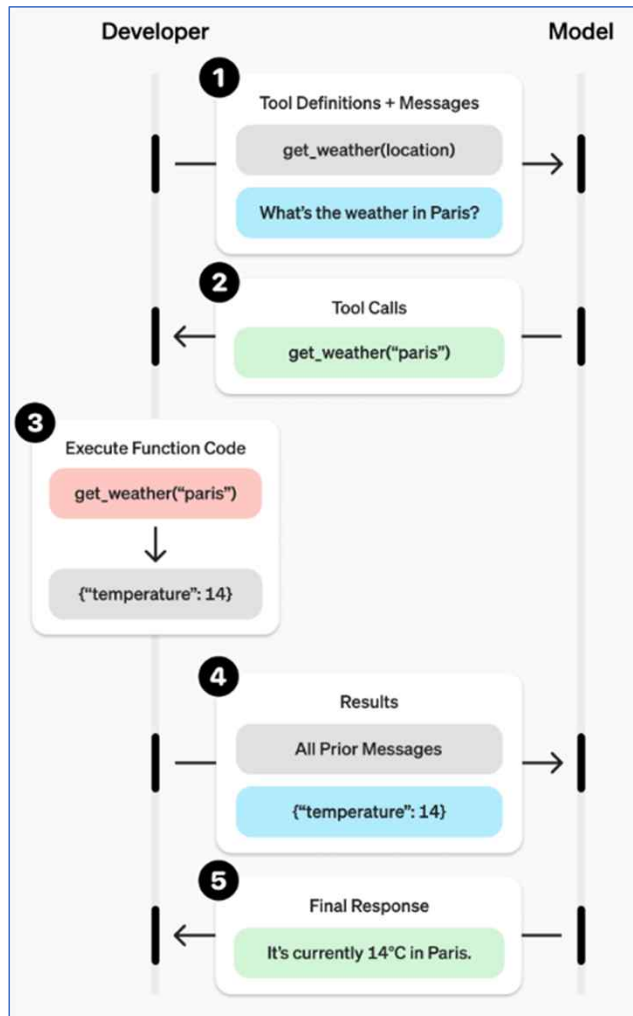
- response_format 파라미터에 JSON 스키마를 지정하여 모델의 최종 응답 자체를 구조화
 - 동작 방식
 - json_schema 타입으로 스키마를 정의하면 모델이 해당 구조를 항상 준수
 - 도구 실행 없이 사용자에게 보여줄 답변을 정해진 형식으로 받을 때 사용
 - 활용 예
 - 문서에서 이름·날짜·금액 등 정형 데이터 추출
 - 수학 풀이 단계, 분류 결과 등 일관된 UI 표시용 응답 생성
 - 함수 호출과의 차이: 함수 호출은 도구에 전달할 "인자"를, 응답 형식은 사용자에게 전달할 "최종 응답"을 구조화
-

실습: 210_구조화된 출력 (Structured Outputs)

- pydantic.BaseModel을 사용한 JSON 스키마 정의
 - 체인 오브 소트(Chain of Thought, COT) 기반 수학 문제 풀이
 - PDF에서 텍스트 추출 및 연구 논문 데이터 구조화
 - 구조화된 출력을 활용한 콘텐츠 심사(Content Compliance)
-

함수 호출 (Function Calling)

- OpenAI 모델이 사용자 코드 또는 외부 서비스와 상호 작용할 수 있도록 지원하는 기능. 주요 기능은 다음 두가지.
 - 데이터 검색
 - RAG 기법을 활용하여 지식 기반에서 데이터를 검색
 - API를 호출하여 실시간 데이터를 가져올 수 있음 (예: 현재 날씨, 주식 가격 등)
 - 액션 수행
 - 예) DB update, API 호출, UI 변경 등
 - 대화형 AI 에이전트가 특정 작업을 실행할 때 활용
-



• 함수 호출 단계

- (1) 함수 정보와 함께 LLM 모델 호출
- (2) 모델이 함수 호출을 결정 – 함수명과 argument 반환
- (3) 함수 호출 – 모델이 만들어준 argument 이용
- (4) 함수 호출 결과를 모델에 제공
- (5) 모델이 최종 응답 생성

실습: 220_함수 호출

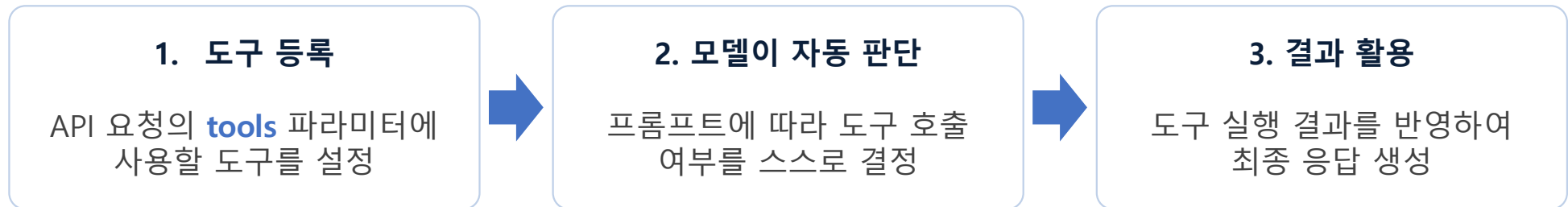
- 기본적인 정보 조회 함수 구현
 - 사용자 요청을 처리할 함수 호출 설정 (`tools`)
 - API를 통해 함수 호출 요청 및 응답 처리
 - 함수 호출 후 JSON 파싱 및 실제 함수 실행
 - 모델 재호출 및 응답 요약
 - 실시간 날씨 데이터 가져오기 (`get\_weather`)
 - AI를 이용한 이메일 보내기
-

Tools

Tools (도구)란?

Tools = LLM이 학습된 지식만으로는 할 수 없는 작업까지 수행하도록 **모델의 능력을 확장하는 기능**
웹 검색 · 파일 검색 · 사용자 정의 함수 호출 · 외부 서비스(MCP) 연동 등을 모델에 연결

동작 방식



사용 예시

```
response = client.responses.create(  
    model="gpt-5.5",  
    tools=[{ "type": "web_search" }],  
    input="오늘의 긍정적인 뉴스 알려줘"  
)
```

1. 사용할 도구 등록
2~3. 모델이 web_search 호출 후 응답 생성

OpenAI 제공 Tools

도구	설명
Function calling	커스텀 코드를 호출하여 모델에 추가 데이터 및 기능 제공
Web search	인터넷 데이터를 모델 응답 생성에 포함
Remote MCP servers	MCP 서버를 통해 모델에 새로운 기능 추가
Skills	버전 관리된 스킬 번들을 호스팅 환경에 업로드하여 재사용
Shell	호스팅 컨테이너 또는 로컬 런타임에서 셸 명령 실행
Computer use	모델이 컴퓨터 인터페이스를 제어하는 에이전틱 워크플로우 생성
Image generation	GPT Image를 사용하여 이미지 생성 또는 편집
File search	업로드된 파일 내용을 검색하여 응답 생성 시 컨텍스트로 활용
Tool search	관련 도구를 동적으로 로드하여 토큰 사용량 및 지연시간 최적화

File Search / Retrieval (파일 검색/검색)

- 파일 검색은 Responses API에서 제공되는 도구
 - 모델이 응답을 생성하기 전에 시맨틱 및 키워드 검색을 통해 이전에 업로드된 파일의 지식 기반에서 정보를 검색
 - 벡터 저장소를 생성하고 파일을 업로드. 모델이 이러한 지식 기반 또는 벡터 저장소에 접근할 수 있도록 하여 모델의 내재된 지식을 강화
 - 이 도구는 OpenAI에서 관리하는 호스팅 도구이므로, 실행을 처리하기 위해 코드를 직접 구현할 필요가 없다. 모델이 이 도구를 사용하기로 결정하면 자동으로 도구를 호출하여 파일에서 정보를 검색하고 출력을 반환.
-

Vector Store 란 ?

- 문서를 벡터화하여 저장하고 검색할 수 있도록 설계된 데이터베이스
 - 청크 분할(800토큰) 및 벡터(임베딩) 저장을 자동으로 수행
 - 키워드 + 벡터 검색을 조합하여 최적의 의미론적 검색 결과 반환
 - Responses API와 연계하여 벡터 검색 결과 활용
 - 하나의 vector store에 최대 10,000 개 파일 저장 가능 (파일당 512MB)
-

MCP (Model Context Protocol)란?

AI 모델을 외부 데이터 · 도구 · 서비스와 연결하기 위한 **개방형 표준 프로토콜**
하나의 표준 규약으로 다양한 시스템을 연결 — 도구마다 개별 연동 코드를 만들 필요가 없음

핵심 구성 요소

Server / Client

서버는 기능을 제공,
클라이언트(Cursor,
Claude Desktop 등)가
이를 사용

Resources

모델에게 필요한 정보
와 컨텍스트를 제공

Tools

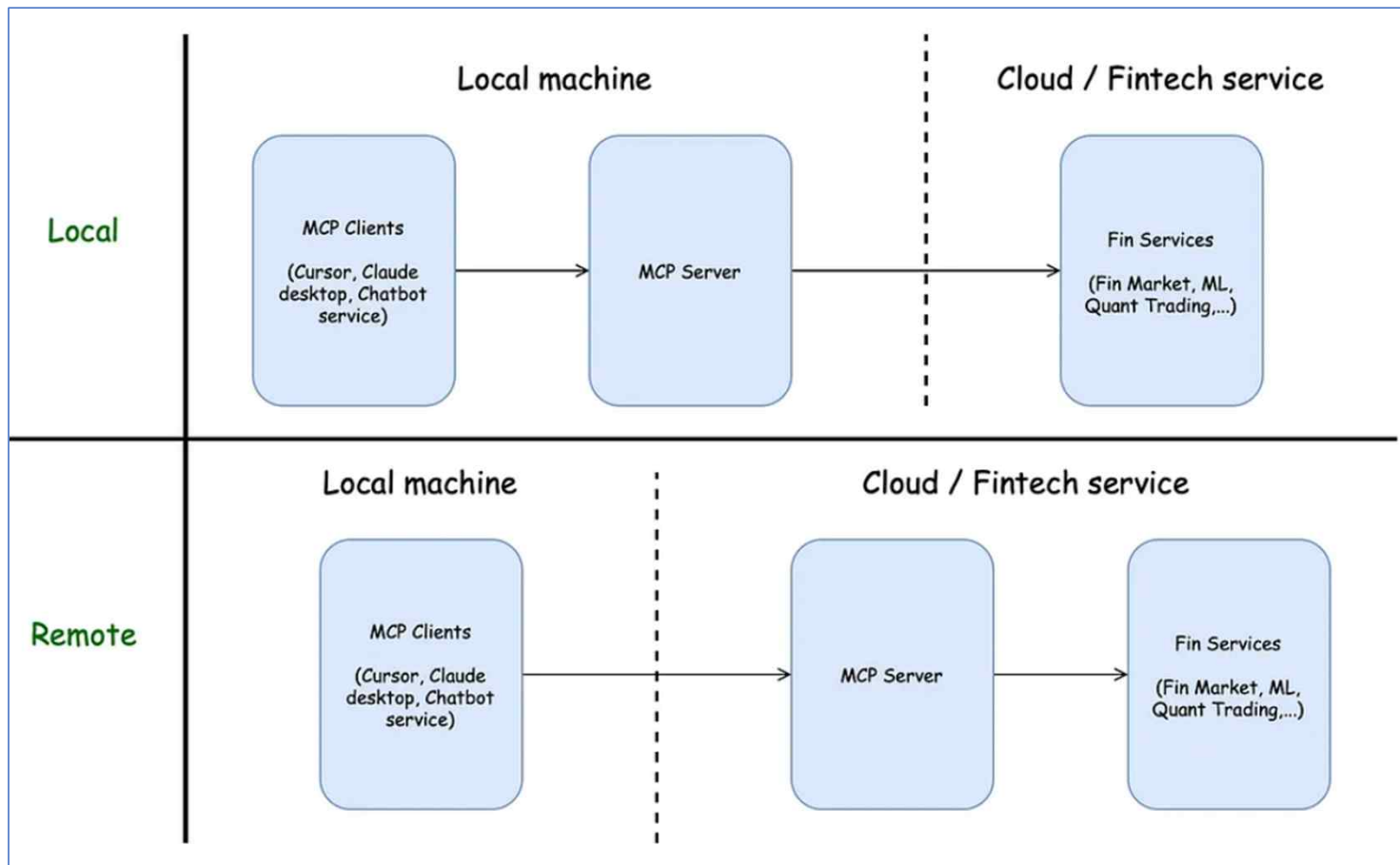
모델이 외부 시스템에
서 작업을 수행할 수
있도록 도구 정의

Prompts

리소스·도구를 활용하
는 재사용 가능한 프롬
프트 템플릿

OpenAI API에서는 `tools=[{ "type": "mcp", "server_url": ... }]` 로
원격 MCP 서버를 등록하면 모델이 해당 서버의 도구를 바로 활용

Local/Remote MCP



Stdio (Standard In/Output)

SSE (Server-Sent Events)

실습: 225_Tools

Tools (도구)

OpenAI 플랫폼에서 사용할 수 있는 도구에 대한 개요는 다음과 같습니다.

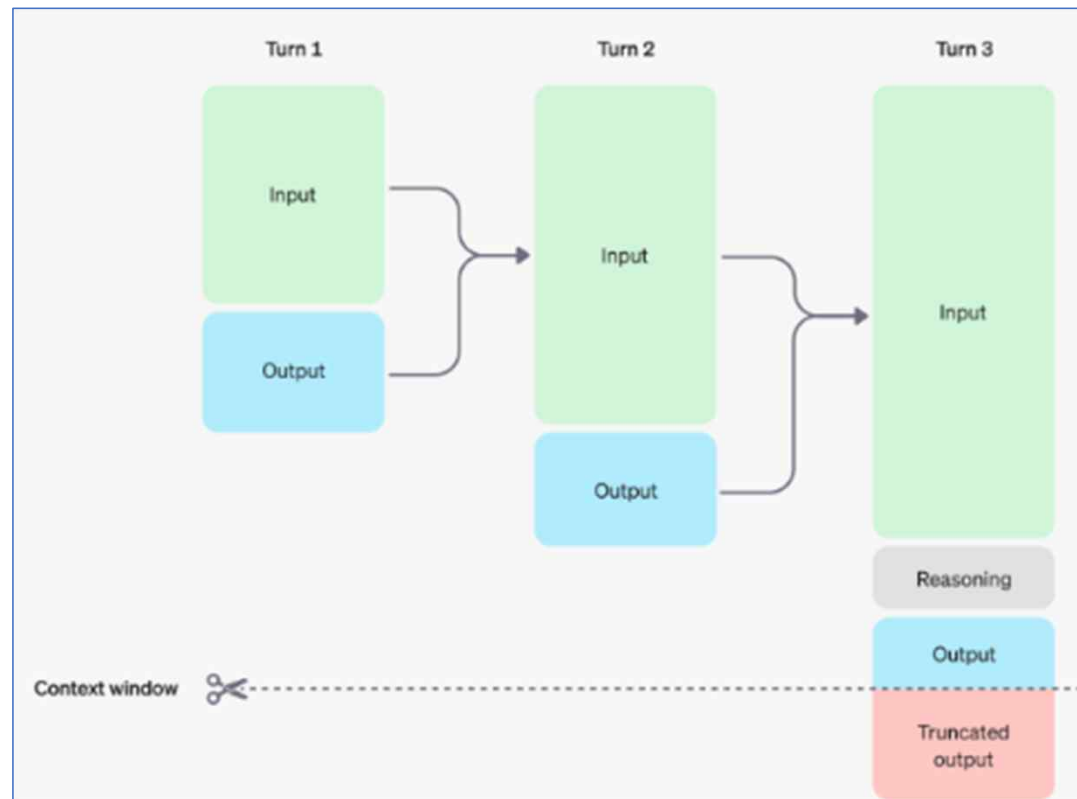
도구	설명
Function calling	커스텀 코드를 호출하여 모델에 추가 데이터 및 기능 제공
Web search	인터넷 데이터를 모델 응답 생성에 포함
Remote MCP servers	MCP 서버를 통해 모델에 새로운 기능 추가
Skills	버전 관리된 스킬 번들을 호스팅 환경에 업로드하여 재사용
Shell	호스팅 컨테이너 또는 로컬 런타임에서 셸 명령 실행
Computer use	모델이 컴퓨터 인터페이스를 제어하는 에이전틱 워크플로우 생성
Image generation	GPT Image를 사용하여 이미지 생성 또는 편집
File search	업로드된 파일 내용을 검색하여 응답 생성 시 컨텍스트로 활용
Tool search	관련 도구를 동적으로 로드하여 토큰 사용량 및 지연시간 최적화

내장 도구 비용

- 참고 : [Built-in tools Pricing](#)

대화 상태 (Conversation State) 유지

- 대화 상태 유지 방법
 - 수동 관리 – memory 상에 대화 내용을 누적하여 저장
 - Response API 이용
- Context 창 관리



Streaming API

- 전체 응답을 생성하는 동안 모델 출력의 시작 부분을 인쇄하거나 처리 가능
 - 완전한 응답이 준비되기 전에도 출력을 점진적으로 표시함으로써 스트리밍은 사용자 경험(UX)을 크게 향상시키며, 특히 LLM의 지연 시간을 처리할 때 더욱 그렇다.
 - 응답 스트리밍은 **stream=True** 설정으로 시작
-

실습 : 230_Conversation State 관리

- 대화 상태 수동 관리 방법
 - Response API를 이용한 대화 상태 관리
 - 챗봇 만들기 : 문서/텍스트 기반의 챗봇 답변 유도
 - Streaming API 를 사용한 UI 개선
-

Agents SDK

LLM 앱의 진화

- 정적인 시스템에서 지능형 시스템으로 진화

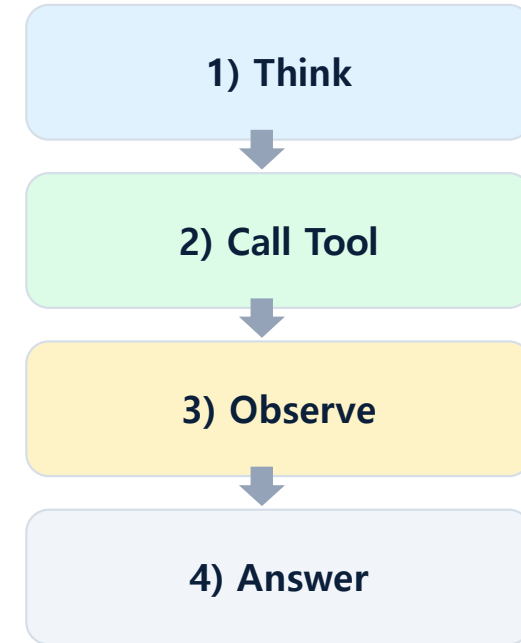
Chatbot → RAG App → RAG + MCP → Agentic AI

- LLM은 다음이 불가능
 - 실시간 데이터 접근
 - 파일 열기
 - 개인 문서 이해
-

Agents: ReAct 루프의 핵심

Agent 란 ?

- LLM을 핵심 엔진으로 사용하여 주어진 목표를 달성하기 위해 독립적으로 작업(추론, 판단, 실행, 피드백)을 수행하는 인공지능 시스템
- 주로 LLM(대규모 언어 모델)의 능력을 기반으로 동작하며, 사용자의 명령을 이해하고, 판단하며, 실행
- 즉, LLM은 두뇌, Agent는 이 두뇌를 활용해 행동을 실행하는 작업자 역할
- 활용 사례
 - 고객 지원 챗봇: 전자상거래 플랫폼에서 환불 요청을 처리하거나 배송 상태 확인
 - 연구 도우미: 과학 논문에서 특정 주제에 대한 핵심 내용 추출
 - 개인 비서: 이메일 초안을 작성하고, 미팅 일정 조율
 - 데이터 분석 에이전트: 주식 시장 데이터를 분석해 투자 전략 제안
 - 교육용 튜터: 수학 문제 풀이 과정을 단계별로 설명



OpenAI Agents 구성 요소

OpenAI Agents SDK 핵심 구성 요소

- **모델 (Model)** : 고급 추론 능력과 에이전트 기능 내재 모델
- **에이전트 (Agents)** : LLM에 지시문과 도구를 장착한 실행 단위
- **핸드오프 (Handoffs)** : 특정 작업을 다른 에이전트에게 위임할 수 있도록 해 줌
- **가드레일 (Guardrails)** : 에이전트의 입력과 출력을 검증하는 안전장치
- **도구 (Tools)** : 함수, MCP, 호스팅 도구 등 에이전트가 행동하게 하는 다양한 도구
- **세션 (Session)** : 에이전트 실행 반복 과정(Agent Loop) 동안 필요한 작업 맥락을 유지해주는 영속적 메모리 레이어
- **추적 (Tracing)** : 에이전트 실행에 대한 내장 추적 기능
- **최신 추가** : Sandbox Agents(컨테이너 기반 장기 작업) · Human-in-the-loop(인간 개입) · Realtime Agents(음성 에이전트)

핸드오프 (Handoffs)

핸드오프 (Handoffs) 란?

- 에이전트가 다른 에이전트에게 작업을 위임하는 기능
- 각 에이전트가 서로 다른 전문 영역을 담당할 때 유용
- 예: 고객 지원 앱에서 **주문 조회**, **환불**, **FAQ** 등을 각각 전담하는 에이전트 구성

핵심 동작 원리

- Handoff는 LLM에게 **도구(Tool)**로 표현됨
- "Refund Agent"로의 핸드오프 → LLM이 **transfer_to_refund_agent** 도구 호출

가드레일 (Guardrails)

가드레일 (Guardrails) 이란?

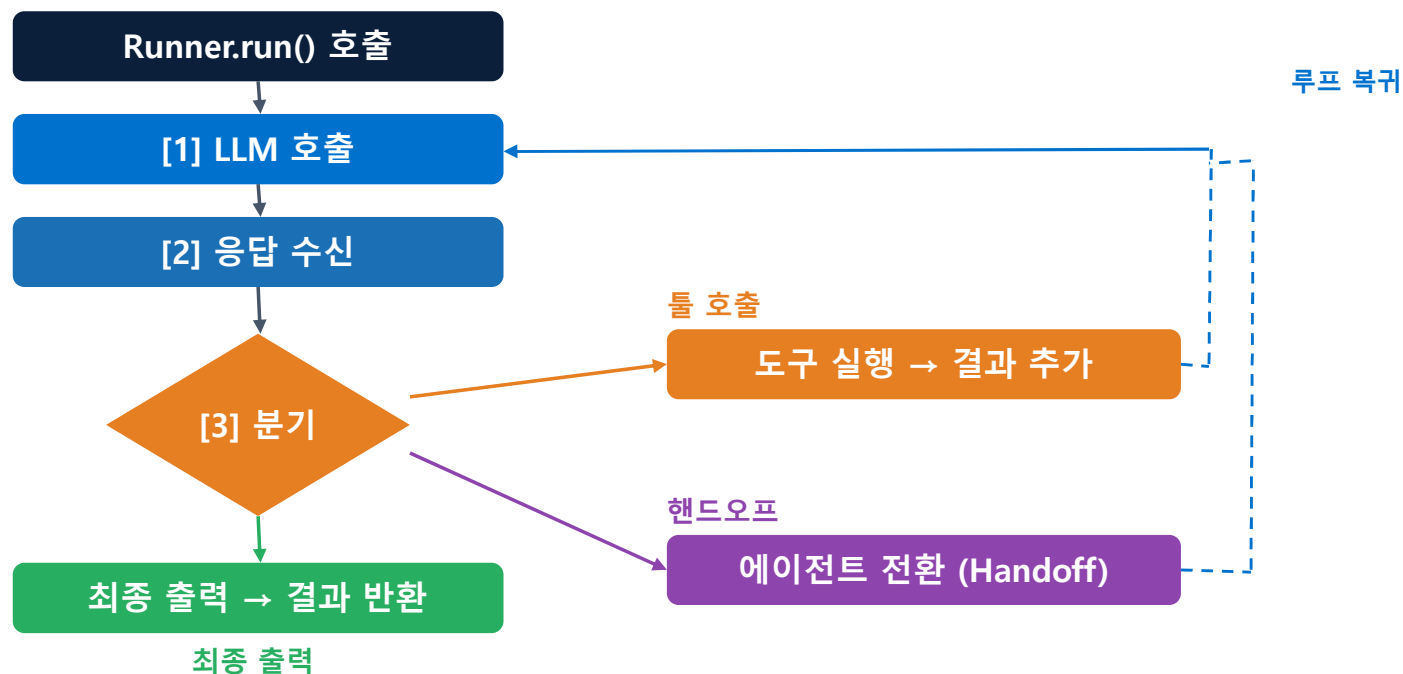
- 에이전트와 병렬로 실행되며, 사용자 입력과 에이전트 출력에 대해 검사와 유효성 검증을 수행
- 빠르고 저렴한 모델을 활용하여 입력이 악의적인 목적인지 먼저 검사
- 의심스러운 입력 감지 시 즉시 오류를 발생시켜 고비용 모델 실행을 차단
- 시간과 비용을 절약하는 입력·출력 보호 장치

예시 시나리오

- 똑똑하지만 느리고 비용이 많이 드는 모델을 사용하는 에이전트를 악의적 사용자가 수학 숙제 대행 같은 목적에 맞지 않는 요청하는 경우
- 가드레일이 빠르고 저렴한 모델로 먼저 검사 → 감지 → 즉시 차단

Agent Loop 흐름

에이전트 루프 동작 순서



툴 호출/핸드오프 → [1]로 돌아가 LLM 재호출, 최종 출력 시 루프 종료

■ Start

■ [1] LLM

■ [2] 응답

■ 툴 호출

■ 핸드오프

■ 완료

700. OpenAI Agent (Agents Overview)

OpenAI 플랫폼을 사용하여 사용자를 대신해 행동(예: 컴퓨터 제어 등)을 수행할 수 있는 에이전트(**Agent**)를 구축할 수 있습니다. Python용 **Agent SDK**를 사용하면 백엔드에서 이러한 에이전트의 오케스트레이션(조율) 로직을 만들 수 있습니다.

OpenAI Agents SDK는 에이전트 기반 AI 애플리케이션을 개발할 수 있게 해주는 도구입니다.

주요 기능 요약:

- **에이전트 루프 (Agent Loop):**
도구 실행 → 결과 전달 → LLM 호출 → 반복 실행 → 완료까지 자동 처리
- **핸드오프 (Handoffs):**
여러 에이전트 간의 협업과 위임을 유연하게 처리 가능
- **가드레일 (Guardrails):**
에이전트 입력을 사전 검사/검증하여, 조건을 만족하지 않으면 조기 종료 가능

```
# 사전 준비: Agents SDK 설치 (Python 3.10 이상 필요)
pip install openai-agents
pip install 'openai-agents[voice]'    # 음성 지원
```

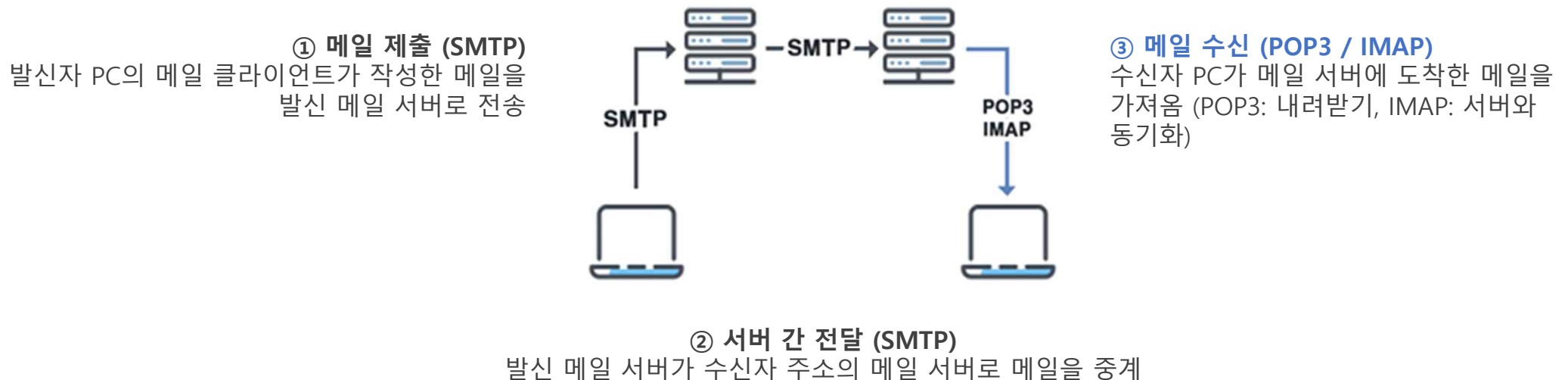
실습 : 701. Chatbot Agent 구현

- Agent 정의
 - 대화 기록 저장
 - 비동기 실행
 - 한 토큰씩 실시간으로 출력되는 "스트리밍" 방식 실행
 - 도구를 추가한 Chatbot Agent 구현
-

부록

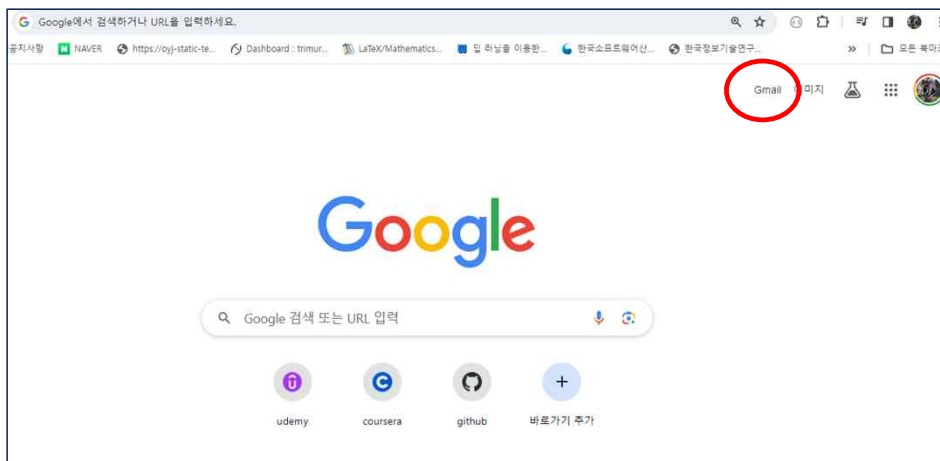
SMTP(Simple Mail Transfer Protocol)

- SMTP는 메일 발송 요청을 처리하는 서버
- SMTP가 메일 발송을 위한 프로토콜이라면, IMAP은 메일 수신을 위한 프로토콜
- MIME(Multipurpose Internet Mail Extensions)은 메일 작성을 위한 표준 형식

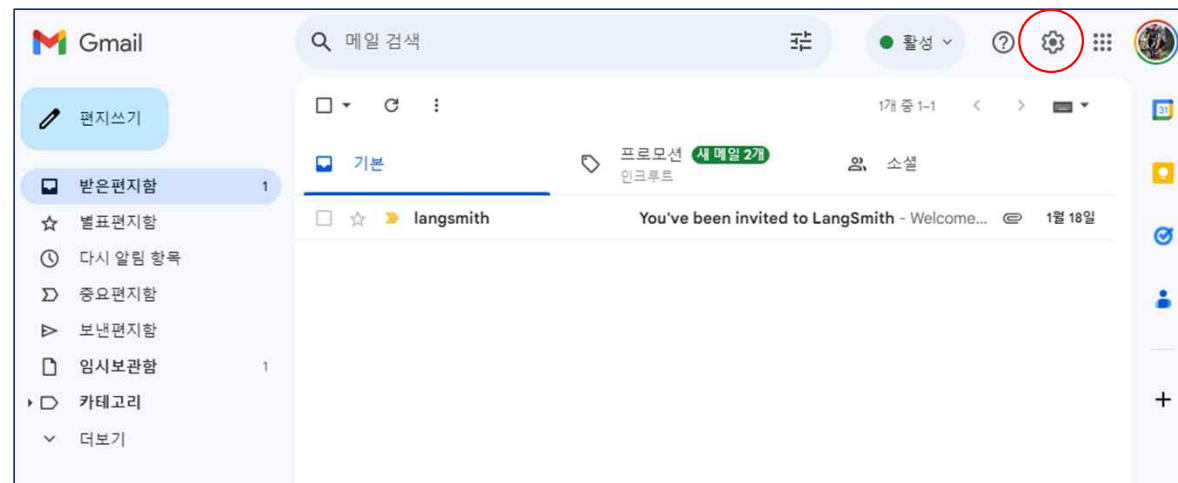


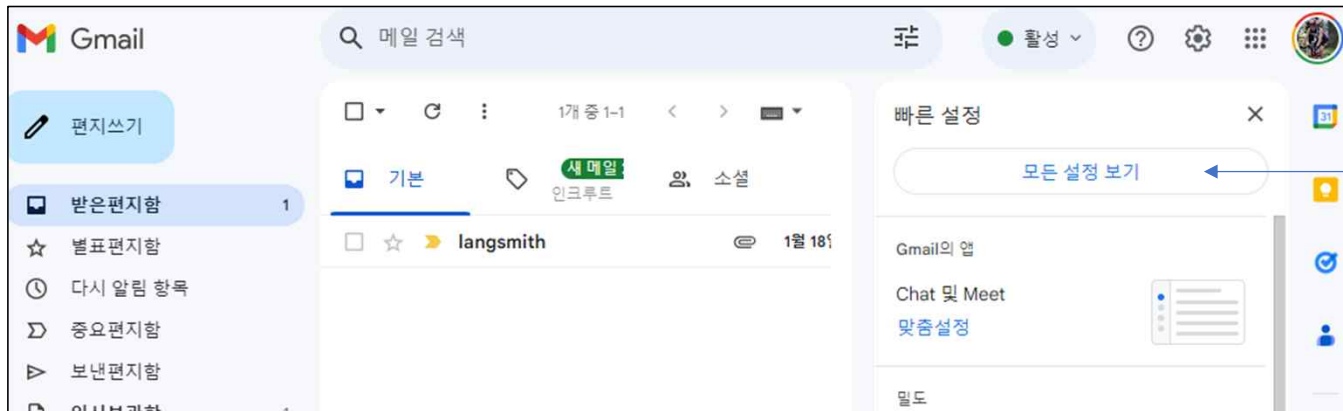
Google SMTP 서버 이용 방법

(1) Gmail 이동



(2) Gmail 설정 화면으로 이동 → 톱니바퀴 모양의 버튼 click





(3) 모든 설정 보기 click



(4) 전달 및 POP/IMAP click

(5) IMPA 액세스 상태를 IMAP 사용으로 변경

IMAP 액세스:
(IMAP를 사용하여 다른 클라이언트에서 Gmail에 액세스)
[자세히 알아보기](#)

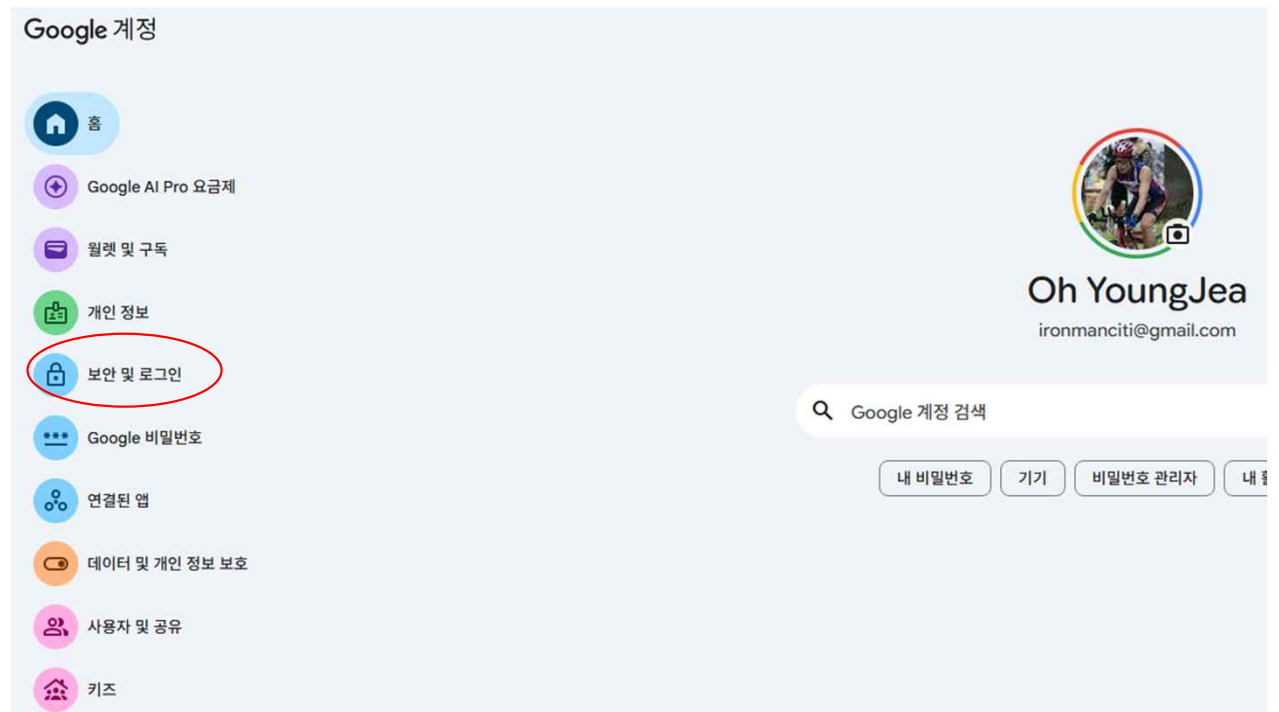
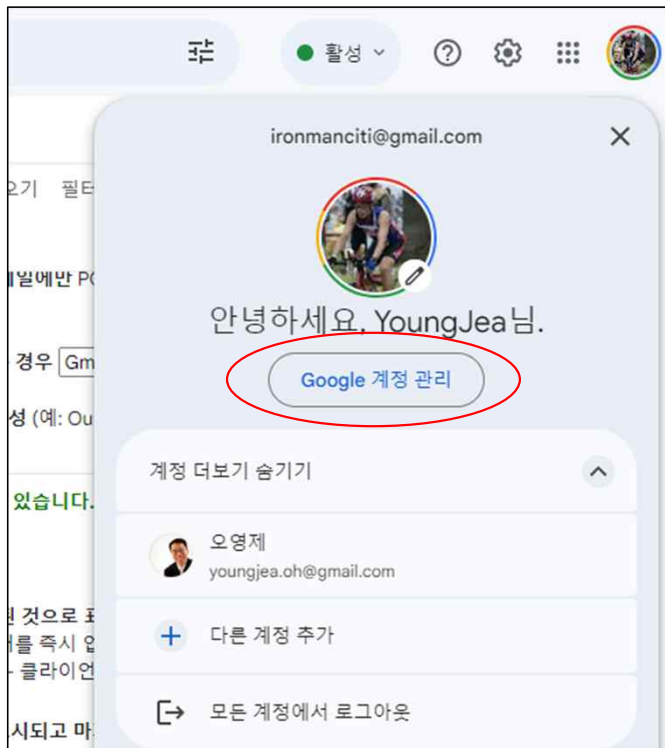
상태: IMAP를 사용할 수 있습니다.
☒ IMAP 사용
☐ IMAP 사용 안함

IMAP에서 메일을 삭제된 것으로 표시하는 경우:
☒ 자동 삭제 사용 - 서버를 즉시 업데이트(기본값)

(6) 변경 사항 저장 click

이메일 클라이언트 구성(예: Outlook, Thunderbird, iPhone)
[설정 방법](#)

(7) Google 계정 보안 설정 화면으로 이동





홈



Google AI Pro 요금제



월렛 및 구독



개인 정보



보안 및 로그인



Google 비밀번호



연결된 앱



데이터 및 개인 정보 보호



사용자 및 공유



키즈

5월 26일

보안 활동 검토

Google에 로그인하는 방법

항상 Google 계정에 액세스할 수 있도록 최신 정보가 반영되었는지 확인하세요.



2단계 인증

✓ 사용 설정일: 2024. 1. 19.



패스키 및 보안 키

패스키 4개



비밀번호

최종 변경일: 2020. 3. 30.



가능한 경우 비밀번호 건너뛰기

✓ 사용



Google 메시지

기기 2대

(8) 2단계 인증 설정

Google에 로그인하는 방법

항상 Google 계정에 액세스할 수 있도록 최신 정보가 반영되었는지 확인하세요.

🔒 2단계 인증	✅ 사용 설정일: 2020. 6. 4.	>
*** 비밀번호	최종 변경일: 2020. 3. 30.	>
📱 Google 메시지	기기 3대	>
☎ 2단계 인증 전화번호	010-5143-6811	>

Click하여 화면 제일 하단의 앱 비밀번호 설정

앱 비밀번호

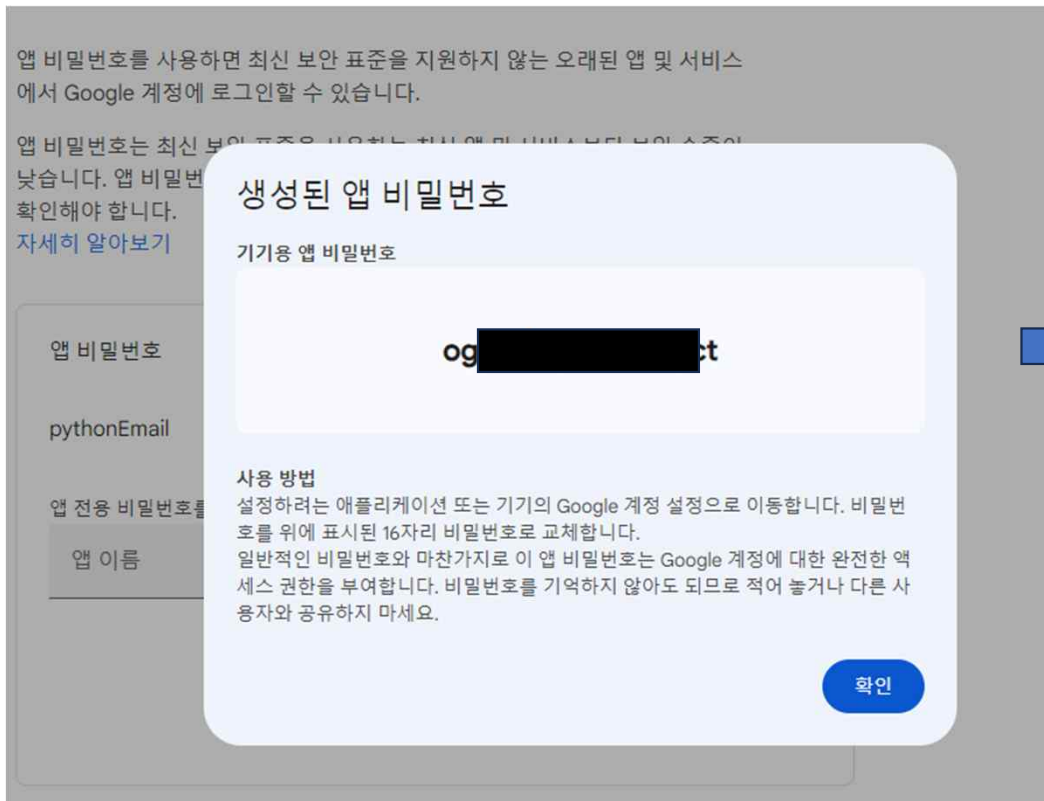
앱 비밀번호는 권장되지 않으며 대부분의 경우 필요하지 않습니다. 계정을 안전하게 보호하려면 'Google 계정으로 로그인'을 사용하여 앱을 Google 계정에 연결하세요.

앱 비밀번호

없음

>

• 앱 비밀번호 생성 (임의의 앱 이름 부여)



생성된 앱 비밀번호를 .env 파일에 저장
smtp_password=sxxx hxxh qxxx nxxx